AWS CERTIFIED CLOUD PRACTITIONER · CLF-C02

The complete course

Every exam topic explained from zero, built on the official AWS exam guide v1.3 and the AWS documentation, with analogies, examples, diagrams and exam tips.

4 domains
following the official task statements 1.1 to 4.3

Explained
analogies, examples, diagrams and notes

Exam ready
traps, key-word map and 30 self-test questions

Contents

Introduction: how this course works · the exam

Part 1 · Cloud Concepts

1.1 The benefits of the AWS Cloud
1.2 Design principles: the AWS Well-Architected Framework
1.3 Migration to the AWS Cloud
1.4 Cloud economics

Part 2 · Security and Compliance

2.1 The shared responsibility model
2.2 Security, governance and compliance concepts
2.3 Access management
2.4 Security components and resources

Part 3 · Cloud Technology and Services

3.1 Deploying and operating in the AWS Cloud
3.2 The AWS global infrastructure
3.3 Compute services
3.4 Database services
3.5 Network services
3.6 Storage services
3.7 AI/ML and analytics services
3.8 Other in-scope services

Part 4 · Billing, Pricing, and Support

4.1 Pricing models
4.2 Billing, budgets and cost management
4.3 Technical resources and AWS Support

Part 5 · Exam day

5.1 How to answer
5.2 Key word → answer
5.3 Last-hour checklist
5.4 Test yourself

How this course works

This course is built in two layers.

1. **The base is the official AWS exam guide (version 1.3, 29 October 2026).** Every section of Parts 1 to 4 is one official *task statement* (1.1 to 4.3), in the same order as the guide.
2. **On top of it comes the explanation**: plain-language explanations, everyday comparisons, examples, diagrams and practical tips, built from the official AWS documentation, whitepapers and service pages.

You do not need to know anything about the cloud to start. Read one part per sitting, take your time with the diagrams, and do the 30 questions at the end.

The boxes you will meet

> ANALOGY
> An everyday comparison to make an idea stick.

> EXAMPLE
> A concrete situation where the concept is used.

> NOTE
> A term or a detail explained in more depth.

> GOOD TO KNOW
> A practical tip or detail worth remembering.

> ON THE EXAM
> How questions are asked, and the traps.

> OUR STORY THROUGH THIS COURSE
> A small bakery, Sara's Bakery, follows us through the course: every concept is applied to her online shop.

Tags: NEW new or renamed in v1.3 · LEGACY only in older guides, know one line · OUT out of scope, just recognize the name.

The exam

Questions	65 in total: 50 count, 15 are unscored test questions you cannot identify
Question types	Multiple choice (1 correct out of 4) and multiple response (2 or more correct out of 5 or more)
Time	90 minutes of exam, about 120 minutes in total with check-in verify
Score	Scaled from 100 to 1000. **You pass with 700.** The score is global: you can be weak in one domain if you are strong elsewhere. A wrong answer costs nothing, so never leave a blank
Where	Pearson VUE test center or online from home with a proctor (webcam, clean desk)
Validity	3 years

Domain	Weight	What it is about
1 Cloud Concepts	24%	Why the cloud, its benefits, good design, migration, economics
2 Security and Compliance	30%	Who protects what, identities and access, security services, compliance
3 Cloud Technology and Services	34%	The services: compute, storage, databases, network, AI, and others
4 Billing, Pricing, and Support	12%	How AWS charges, how to control costs, support plans

The exam does not ask you to code or configure anything. It asks: **which service or concept fits this situation?** If you understand what each service is *for*, you pass.

PART 1 · DOMAIN 1

Cloud Concepts

24% of the scored content

What the cloud is, why companies use it, what makes a good cloud architecture, how a company moves to the cloud, and how the cloud changes the money side of IT.

IN THIS PART

1.1 Define the benefits of the AWS Cloud

1.2 Identify design principles of the AWS Cloud

1.3 Understand the benefits of and strategies for migration to the AWS Cloud

1.4 Understand concepts of cloud economics

OUR STORY THROUGH THIS COURSE

Sara owns a small bakery, *Sara's Bakery*. She wants to sell cakes online. We will follow her as she builds her website, protects it, grows it and pays for it. Every time a new idea appears, ask yourself: what would it change for Sara?

1.1 The benefits of the AWS Cloud

Step 1: what is a server?

IN PLAIN WORDS

When you open a website, your phone or laptop sends a request over the internet to another computer somewhere in the world. That computer prepares the page and sends it back. This computer is called a **server**, because it *serves* answers to others.

A server is a computer like yours, with a processor (**CPU**), memory (**RAM**) and disks, but built to run **24 hours a day**, to answer **many people at once**, and usually without a screen or keyboard. You manage it remotely.

Every application you know runs on servers: Instagram, your bank's app, Netflix, an online shop. Even a tiny website needs at least one.

> NOTE
> Three words you will meet everywhere. **Compute** = the processing power that runs programs (CPU and RAM). **Storage** = the space where data is kept (disks). **Network** = the connections that let computers talk to each other and to the internet.

Step 2: how companies ran servers before the cloud

Before the cloud, a company that needed servers had to **own** them. This is called **on-premises** (often shortened to **on-prem**): the hardware is on the company's premises.

Let us follow Sara in the old world. To open her online shop she must:

1. **Buy servers** (several thousand euros) and wait weeks for delivery.
2. Find a **room** for them with air conditioning (servers produce heat), a reliable electricity supply, maybe a backup generator, and a fast internet connection. A room built for this is a **data center**.
3. **Install** everything: operating system, web software, database, security.
4. **Hire someone**, or learn herself, to maintain it: updates, repairs, backups, replacing broken disks at 3 a.m.
5. **Guess** how much power she needs for the next three to five years.

That last point is the worst. If she buys too little, her site crashes on Mother's Day when everyone orders cakes. If she buys too much "just in case", the machines sit idle 360 days a year and the money is wasted. And she pays everything **upfront**, before selling a single cake online.

> ANALOGY
> Running your own servers is like generating your own electricity with a generator in the garden. You buy it, maintain it, refuel it, and if your needs grow you buy a bigger one. The cloud is like plugging into the electricity grid: someone else runs the power station, you use what you need and pay for what you consume.

Step 3: what cloud computing is

AWS gives this official definition, worth learning word for word:

> NOTE
> **Cloud computing is the on-demand delivery of IT resources over the internet with pay-as-you-go pricing.**

Let us take it apart:

- **On-demand**: you get resources **when you ask for them**, in minutes, by clicking or by running a command. No ordering hardware, no waiting weeks.

- **IT resources**: servers, storage, databases, networks, and also ready-made services such as email sending, artificial intelligence or video conversion. AWS offers more than 200 services.
- **Over the internet**: you reach and manage everything remotely, from a browser or a program. The data centers are AWS's, not yours.
- **Pay-as-you-go**: you pay **only for what you use**, often by the second or by the gigabyte, with **no upfront payment** and no long contract. When you delete a resource, you stop paying.

Back to Sara in the cloud world. She opens an AWS account with a credit card, chooses a server in a few clicks, and her shop is online the same afternoon. On Mother's Day she adds two more servers for the rush, then removes them the next day. Her bill follows her activity: small when sales are quiet, larger when business is good. She never sees a cable or a fan.

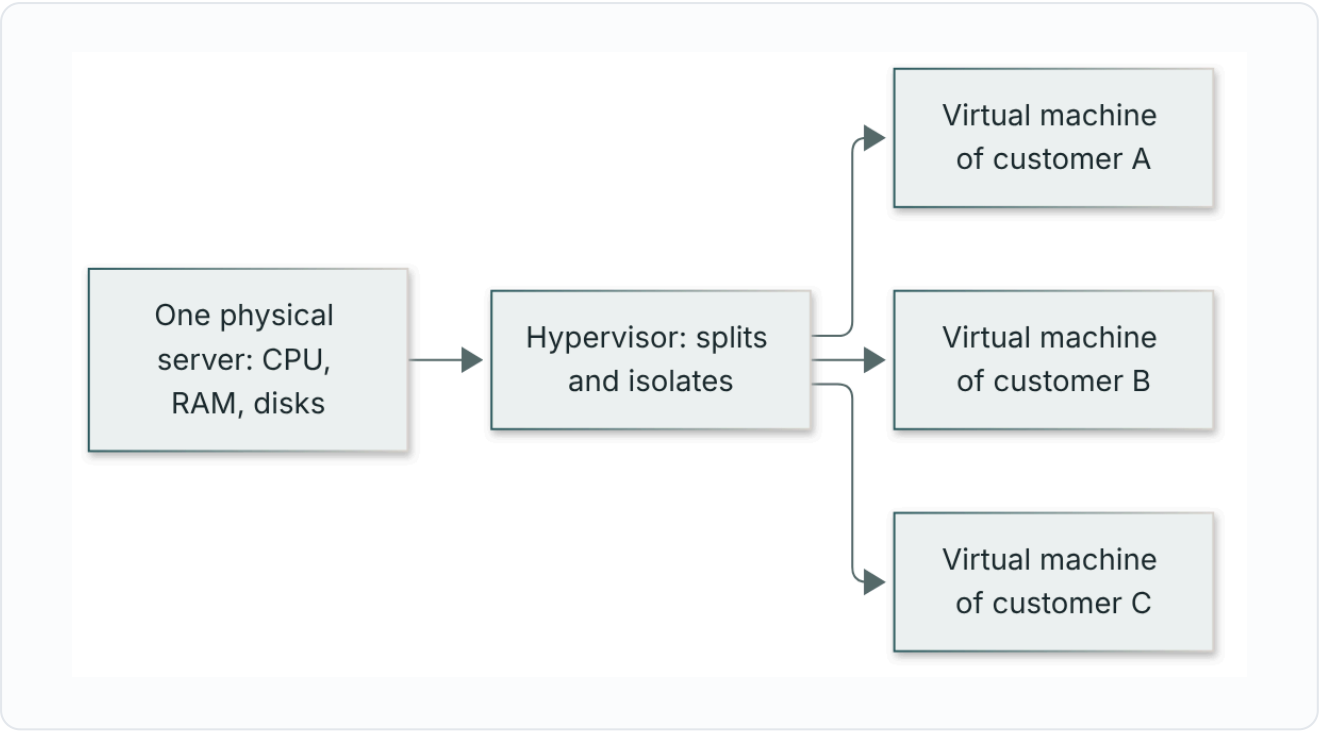
Who is AWS? Amazon Web Services is the cloud division of Amazon. Amazon built huge computing skills to run its own online shop, then started renting them to others. Its very first public service was **Amazon SQS** (a message queue) in 2004, followed in 2006 by **Amazon S3** (storage) and **Amazon EC2** (virtual servers). AWS is the largest cloud provider in the world; Microsoft Azure and Google Cloud are the other two big ones.

> GOOD TO KNOW
> A real **cloud service provider (CSP)**, and not just a hosting company, offers dozens or hundreds of services that can be **combined** with each other, all reachable through **one single API** (the programmatic interface), **metered billing** (per second, per GB, per request), **built-in monitoring** of every action (on AWS: CloudTrail), **infrastructure services** (compute, storage, network, databases), and **automation through infrastructure as code**.

Step 4: how it works behind the scenes, virtualization

How can AWS give a server to millions of customers in minutes? It does not buy one physical machine per customer. It uses **virtualization**.

A big physical server runs a special program called a **hypervisor**. The hypervisor divides the machine into several **virtual machines (VMs)**. Each virtual machine has its own share of CPU, memory and disk, its own operating system, and behaves exactly like an independent computer. Customers sharing the same physical machine are completely isolated from each other and cannot see each other's data. Sharing hardware between many customers is called **multi-tenancy**.



Over time, the way we run applications evolved, and each step leaves you less work:

Step	What it is	Picture it as	What you still manage
Dedicated server	A whole physical machine just for you	Owning a house	Everything, including the hardware
Virtual machine	A slice of a physical machine, with its own operating system	Renting an apartment in a building	Operating system, software, scaling
Container	A lightweight package with your application and everything it needs, sharing the operating system of the host	Renting a room in a shared flat: private room, shared kitchen	Your containers and where they run
Function (serverless)	You upload only a piece of code; it runs when something happens and stops after	Taking a taxi: you only pay for the ride	Only your code

> **NOTE**
> Serverless functions can have a **cold start**: if a function has not run for a while, the first request waits a little longer while AWS prepares an environment. It is the small price of paying nothing when nothing runs.

Step 5: the three service models, IaaS, PaaS and SaaS

IN PLAIN WORDS

When you use a cloud service, a key question is: **how much does the provider manage for you, and how much is left to you?** The answer defines three models: **IaaS**, **PaaS** and **SaaS**.

To understand them, picture an application as a stack of layers, from the bottom (physical things) to the top (what users see):

1. **Hardware**: the physical servers, disks and network equipment, in a data center.
2. **Virtualization**: the hypervisor that creates virtual machines.
3. **Operating system**: Linux or Windows, which must be installed, configured and **patched** (security updates).
4. **Runtime and middleware**: the software your application needs to run, for example Java, Python, a web server.
5. **Data**: the information your application stores.
6. **Application**: the program itself, the part users see.

The three models simply draw the line between "the provider manages this" and "you manage this" at a different height of the stack.

Layer	On-premises	IaaS	PaaS	SaaS
Application	You	You	You	**Provider**
Data	You	You	You	Provider (you own your data)
Runtime, middleware	You	You	**Provider**	Provider
Operating system	You	You	Provider	Provider
Virtualization	You	**Provider**	Provider	Provider
Hardware, data center	You	Provider	Provider	Provider

IaaS, Infrastructure as a Service. The provider gives you the **basic building blocks**: virtual servers, storage and networks. Everything from the operating system upward is **your job**: installing software, applying updates, configuring security. You get **maximum control and flexibility**, but also the most work. It is the closest to having your own servers, without the hardware.

- AWS examples: **Amazon EC2** (virtual servers), **Amazon VPC** (networks), **Amazon EBS** (virtual disks).
- For Sara: she rents a virtual server on EC2, installs Linux updates herself, installs a shop software, and maintains it.
- Choose it when: you need full control, special software, or you move existing servers to the cloud as they are.

PaaS, Platform as a Service. The provider manages the infrastructure **and** the operating system and runtime. You bring **only your application code and your data**. No patching, no server configuration. You lose some control but you gain a lot of time.

- AWS examples: **AWS Elastic Beanstalk** (upload code, AWS creates and runs the servers), **Amazon RDS** (a database where AWS manages the database software), **AWS Lambda** (run

code without servers).

- For Sara: her developer uploads the shop's code to Elastic Beanstalk; AWS handles servers, updates and scaling.
- Choose it when: developers want to focus on writing the application, not on running servers.

SaaS, Software as a Service. The provider gives you a **complete, ready-to-use application**, usually in a web browser. You do not install or maintain anything; you just use it and configure your settings.

- Examples: Gmail, Microsoft 365, Salesforce, Zoom; on AWS: **Amazon WorkSpaces** (virtual desktops), **Amazon Connect** (call center).
- For Sara: she simply subscribes to a ready-made online shop service and adds her cakes and prices.
- Choose it when: the software you need already exists and you do not want to build or run anything.

> ANALOGY: GETTING A PIZZA
> On-premises: you make the dough, the sauce, bake it in your oven and serve it. IaaS: you buy a ready-made pizza base and do the rest at home. PaaS: you order a pizza delivered, and only provide the table and drinks. SaaS: you eat at the restaurant, everything is done for you.

> ANALOGY: GETTING AROUND
> Own car = on-premises (you buy, insure, repair). Rented car = IaaS (no purchase, but you drive and refuel). Taxi = PaaS (you just say where to go). Bus = SaaS (a complete service, you just get on).

> ON THE EXAM
> "Maximum control over the operating system" = IaaS, EC2. "Developers want to deploy code without managing infrastructure" = PaaS, Elastic Beanstalk. "Use a complete application without managing anything" = SaaS. The more you go from IaaS to SaaS, the less you manage, and this also shifts security responsibility (see 2.1).

Step 6: where the cloud runs, the deployment models

IN PLAIN WORDS

A company can run its applications in three ways.

Cloud (also called public cloud, cloud-native or "all-in"): everything runs at a cloud provider like AWS. "Public" does **not** mean your data is public: it means the infrastructure is shared with other customers, while your resources stay private and isolated. Startups and new projects usually start here: nothing to buy, everything available immediately.

On-premises (also called private cloud when it is automated with cloud-like tools such as OpenStack or VMware): everything stays in the company's own data center. Some organizations keep this because of very strict rules, huge past investments, or habits. They keep all the problems we saw in step 2.

Hybrid: a mix of both, **connected together**. Some systems stay on-premises, others run in the cloud, and they exchange data. This is very common in large companies: they cannot move everything at once, or some data must stay in their building.

> EXAMPLE
> A bank keeps its core banking system in its own data center because of regulations, but runs its mobile app and its data analysis on AWS. The two are linked by a private line (**AWS Direct Connect**) or an encrypted tunnel over the internet (**AWS VPN**). That is a hybrid deployment.

AWS services that make hybrid possible: **AWS Direct Connect** and **AWS VPN** (connect your network to AWS), **AWS Storage Gateway** (your local applications use cloud storage), **AWS Outposts** (AWS installs its own hardware inside your data center).

> GOOD TO KNOW
> Typical profiles: startups and SaaS companies are **cloud**; banks, insurers and large firms with old systems are **hybrid**; some public sector organizations or very regulated industries stay largely **on-premises**, although most now use the cloud for at least part of their work.

Step 7: the six advantages of cloud computing

AWS summarizes the value of the cloud in six advantages. The exam uses this exact wording, so learn each sentence, and above all understand what it means.

1. Trade fixed expense for variable expense. Before: Sara pays a large amount upfront for servers and a room, whether she sells or not. This is a **fixed** cost, an investment called **CapEx** (capital expenditure). With the cloud: she pays a monthly bill that follows her real usage. This is a **variable** cost, an operating cost called **OpEx** (operational expenditure). She can start with almost nothing and pay more only when business grows.

2. Benefit from massive economies of scale. AWS buys servers, disks and electricity for millions of customers, so it gets much lower prices than any single company. Because so many customers share the infrastructure, each pays less. AWS has lowered its prices many times as it grew.

3. Stop guessing capacity. Before: you had to guess your needs years in advance, and you were always wrong: too much (wasted money) or too little (crashes). With the cloud you add capacity in minutes when demand rises and remove it when it falls. Sara adds servers for Mother's Day and removes them afterwards.

4. Increase speed and agility. New resources arrive in minutes instead of weeks. Teams can test an idea quickly and cheaply: if it fails, delete everything and the cost was tiny. This encourages innovation.

5. Stop spending money running and maintaining data centers. Racking servers, replacing disks, managing cooling and power is necessary work but does not make a company different from its competitors. AWS calls it **undifferentiated heavy lifting**. In the cloud, AWS does it, and your people focus on your customers and products.

6. Go global in minutes. AWS has data centers on every continent. You can deploy your application close to users in Asia, Europe and America in a few clicks, so everyone gets a fast experience, without building anything abroad.

> NOTE
> The official objective of this section maps directly to these advantages: economies of scale (advantage 2), speed of deployment and global reach (advantages 4 and 6), elasticity (advantage 3).

Step 8: the words you must master

These terms appear in many questions. Take time with each one.

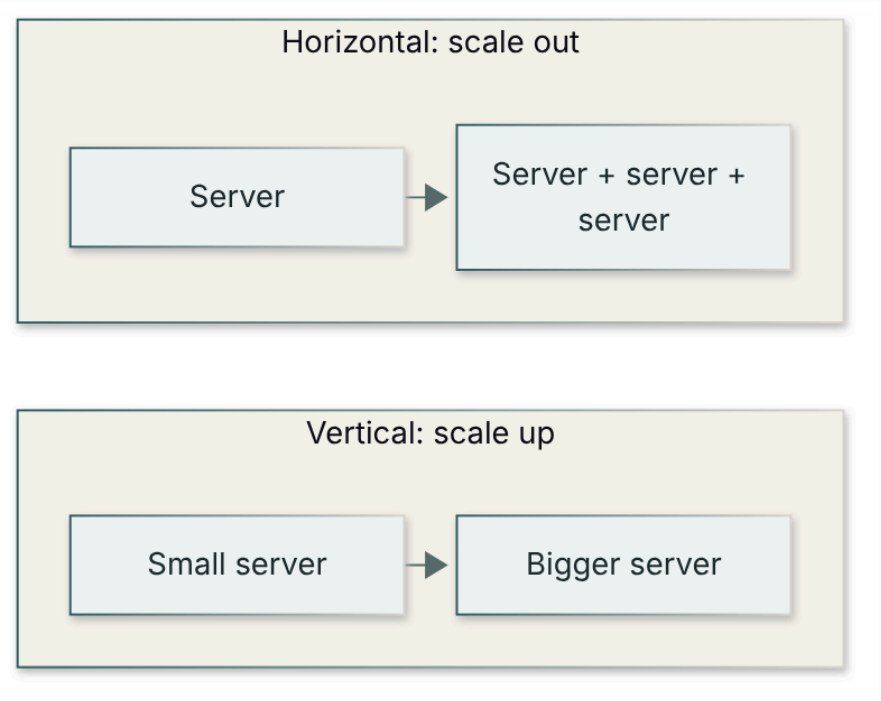
High availability. A system is highly available when it **stays reachable** even if a component fails. The key idea is **no single point of failure**: nothing whose failure alone brings everything down. On AWS you get it by running copies in **several Availability Zones** (separate data centers, see 3.2).

> EXAMPLE
> Sara runs her shop on two servers in two different data centers, with a load balancer in front that sends visitors to whichever server is healthy. A power cut hits one data center: the load balancer sends everybody to the other server. Customers keep ordering cakes.

Fault tolerance. A step further than high availability: the system **keeps working without any interruption** when a component fails, because every part is duplicated and takes over instantly. High availability may accept a short interruption; fault tolerance aims for none. A plane with two engines that can fly on one is fault tolerant.

Scalability. The ability of a system to **handle more load**. There are two ways to scale:

- **Vertical scaling (scale up)**: make one server bigger (more CPU, more memory). Simple, but there is a maximum size, and it often requires a restart.
- **Horizontal scaling (scale out)**: add more servers side by side and share the work. Almost no limit, and if one server fails the others continue. This is the cloud way.



Elasticity. Scaling that happens **automatically and in both directions**: resources are added when demand grows **and removed when it drops**, so you never pay for idle capacity. Scalability is the *ability* to grow; elasticity is the *automatic adjustment* to demand. On AWS, elasticity for servers comes from **Amazon EC2 Auto Scaling**.

Agility. How fast you can experiment and deliver new things. In the cloud, getting a new server takes minutes instead of weeks, so a team can try ten ideas in the time it used to take to try one.

Durability and availability of data. Two different promises:

- **Durability**: your data will **not be lost**. Amazon S3 is designed for 99.999999999% durability ("eleven nines"): store 10 million files and you might lose one in 10,000 years.
- **Availability**: your data is **reachable right now** when you ask for it. S3 Standard is designed for 99.99% availability.

A file can be durable (safe) but temporarily unavailable (you cannot reach it for a few minutes).

> **ON THE EXAM**
> "Automatically add and remove resources to match demand" = **elasticity**, provided by **Auto Scaling**. "No single point of failure", "keep running if a data center fails" = **high availability**, by using **multiple Availability Zones**. "Pay only for what you use instead of buying hardware" = **trade fixed expense for variable expense**. Do not confuse durability (not losing data) and availability (being able to access it).

1.2 Design principles: the AWS Well-Architected Framework

What it is and why it exists

IN PLAIN WORDS

Building in the cloud is easy; building **well** is harder. A system can work on day one and still be fragile, insecure or too expensive. Over many years AWS has reviewed thousands of customer architectures and written down what good ones have in common. The result is the **AWS Well-Architected Framework**: a set of best practices and questions to check that a system (AWS calls it a **workload**) is built well.

It is organized in **six pillars**. Each pillar looks at the system from one angle and asks one big question: does it run well, is it protected, does it survive failures, does it use the right resources, is there waste, and is its impact on the planet low?

The six pillars, one by one

1. Operational excellence: run the system well and keep improving. How you deploy, monitor and operate your application day to day, and how you learn from problems. Main ideas:

- **Perform operations as code**: describe infrastructure and procedures in code and scripts rather than doing them by hand, so they are repeatable and less error-prone.
- **Make frequent, small, reversible changes**: small changes are easy to test and easy to undo if something goes wrong.
- **Anticipate failure** and **learn from all operational failures**: practice incidents, write down what happened, improve.

Sara's case: every update of her shop goes through an automated, tested procedure instead of someone copying files manually at night.

2. Security: protect information, systems and assets. Main ideas:

- **Implement a strong identity foundation**: control who can do what, with the **least privilege** (only the permissions needed).
- **Enable traceability**: record who did what and when (logs), and react to alerts.
- **Apply security at all layers**: network, servers, application, data.
- **Protect data in transit and at rest**: encrypt it while it moves and while it is stored.
- **Prepare for security events**: have a plan for incidents.

Sara's case: the customer database is encrypted, only her developer can access it, and every access is logged.

3. Reliability: work correctly and recover from failures. Main ideas:

- **Automatically recover from failure**: monitor the system and repair or replace failed parts without human action.
- **Test recovery procedures**: do not just hope backups work, test them.
- **Scale horizontally** to increase availability: several small servers rather than one big one.
- **Stop guessing capacity**: let the system scale with demand.
- **Manage change through automation**.

Sara's case: her shop runs in two data centers and her database has an automatic standby copy.

4. Performance efficiency: use computing resources efficiently. Main ideas:

- **Democratize advanced technologies**: instead of building complex technology yourself (machine learning, databases), use AWS managed services that make it available to everyone.
- **Go global in minutes**: deploy close to users.
- **Use serverless architectures**: no servers to run, AWS adapts the capacity.
- **Experiment more often** and choose the right type of resource for each job.

Sara's case: her cake photos are delivered from servers close to each visitor, so pages load fast everywhere.

5. Cost optimization: deliver value at the lowest price. Main ideas:

- **Adopt a consumption model**: pay only for what you use, switch off what you do not use (test servers at night).
- **Measure overall efficiency** and **analyze and attribute expenditure**: know which team or project spends what.
- **Stop spending money on undifferentiated heavy lifting**: let AWS run the infrastructure.

Sara's case: her test server stops automatically every evening, and her main server is the right size, not oversized.

6. Sustainability: reduce the environmental impact. Main ideas: understand your impact, **maximize utilization** (busy servers waste less energy than idle ones), use **managed services** and the most efficient hardware, reduce the data you keep and move.

General design principles that apply to all pillars: stop guessing your capacity needs, test systems at production scale, automate to make experimentation easier, allow architectures to evolve, drive decisions with data, and practice through **game days** (simulated incidents).

Pillar	The question it answers	Key words in a question
Operational excellence	Do we run and improve the system well?	operations as code, small reversible changes, runbooks, learn from failures

Pillar	The question it answers	Key words in a question
Security	Are data and systems protected?	least privilege, traceability, encryption, all layers
Reliability	Does it recover and meet demand?	recover from failure, test recovery, Multi-AZ, scale horizontally, stop guessing capacity
Performance efficiency	Do we use the right resources efficiently?	right instance type, serverless, go global, democratize technologies
Cost optimization	Are we wasting money?	pay for use, rightsizing, Spot, turn off idle resources
Sustainability	Is the environmental impact minimal?	utilization, efficient hardware, carbon footprint

Tools. The **AWS Well-Architected Tool** is a free service in the AWS console: you describe your workload, answer questions pillar by pillar, and get a list of risks and recommendations. The **AWS Architecture Center** is a website with reference architectures and diagrams for common use cases.

NOTE

Pillars overlap, and the exam asks for the **best** match. Ask yourself what the question is mainly worried about: losing service = Reliability; paying too much = Cost optimization; being slow = Performance efficiency; being attacked or leaking data = Security; daily operations and improvement = Operational excellence; energy and carbon = Sustainability.

ON THE EXAM

"Stop guessing capacity", "automatically recover from failure" = Reliability. "Perform operations as code", "small reversible changes" = Operational excellence. "Democratize advanced technologies", "use serverless", "go global in minutes" = Performance efficiency. "Enable traceability", "least privilege" = Security.

1.3 Migration to the AWS Cloud

IN PLAIN WORDS

A **migration** is moving existing applications and data from a company's own data center (or another provider) to AWS. For a big company it is a long project: hundreds of applications, thousands of servers, many teams. AWS gives two kinds of help: a framework to **prepare the organization** (the CAF) and a method to **decide what to do with each application** (the 7 Rs), plus tools to do the move.

The AWS Cloud Adoption Framework (AWS CAF)

Why it exists. Most cloud migrations that fail do not fail because of technology. They fail because staff are not trained, nobody owns the budget, security teams block everything, or

managers do not know why they are migrating. The **CAF** is AWS's guide to prepare the **whole organization**, not only the servers.

The CAF looks at the move from **six perspectives**. Each perspective is a group of people in the company with its own concerns:

Business capabilities

- **Business perspective**: makes sure the cloud investment serves the company's goals (growth, new products, savings). People: CEO, finance, business leaders. *Question they ask: why are we doing this and what will we gain?*
- **People perspective**: culture, skills, training, organization of teams, change management. People: HR, managers. *Question: are our people ready and trained?*
- **Governance perspective**: manage the program, the risks, the budget and the benefits. People: CIO, program and project managers, risk and finance. *Question: are we controlling risks and costs?*

Technical capabilities

- **Platform perspective**: build a scalable, reliable cloud environment and architecture. People: CTO, architects, engineers. *Question: what will we build on AWS?*
- **Security perspective**: confidentiality, integrity and availability of data and workloads. People: CISO, security teams. *Question: will it be secure and compliant?*
- **Operations perspective**: run and support cloud services day to day at the agreed level. People: IT operations, support. *Question: who will run it and how?*

The four benefits of the CAF (exact list from the exam guide):

1. **Reduced business risk**: better reliability, security and performance lower the risk of incidents.
2. **Improved environmental, social and governance (ESG) performance**: for example, more energy-efficient infrastructure and better use of data for sustainability goals.
3. **Increased revenue**: new digital products and faster time to market.
4. **Increased operational efficiency**: lower costs, more automation, more productive staff.

> ON THE EXAM
> "Training staff, culture, organizational change" = **People**. "Managing risk, budget, portfolio" = **Governance**. "Aligning the cloud with business goals" = **Business**. "Architecture and infrastructure" = **Platform**. "Protecting data" = **Security**. "Running and supporting services" = **Operations**.

The 7 Rs: what to do with each application

For each application it owns, the company chooses one of seven strategies. Picture moving house: some things go to the dump, some stay behind, some move as they are, some get

repainted, some are replaced by new ones.

Retire. Switch off applications that nobody needs any more. During the inventory, companies often discover old systems still running for nothing. *Saving: immediate.*

Retain. Keep the application where it is, for now. Reasons: it was recently upgraded, it must stay for legal reasons, or it is too complex to move yet. Revisit later.

Rehost, "lift and shift". Move the application **as it is**, without changing its code, from a server in the data center to an EC2 instance. It is the **fastest** strategy with quick savings, and teams can optimize later once in the cloud. Tool: **AWS Application Migration Service**.

Relocate. Move a whole platform without changes, for example an entire VMware environment moved to **VMware Cloud on AWS**. Similar idea to rehost, at the scale of a platform.

Replatform, "lift, tinker and shift". Move the application with **a few optimizations** that bring benefits, without changing its architecture. Classic example: instead of moving the database server as it is, move the data to **Amazon RDS**, so AWS manages the database software, backups and patches.

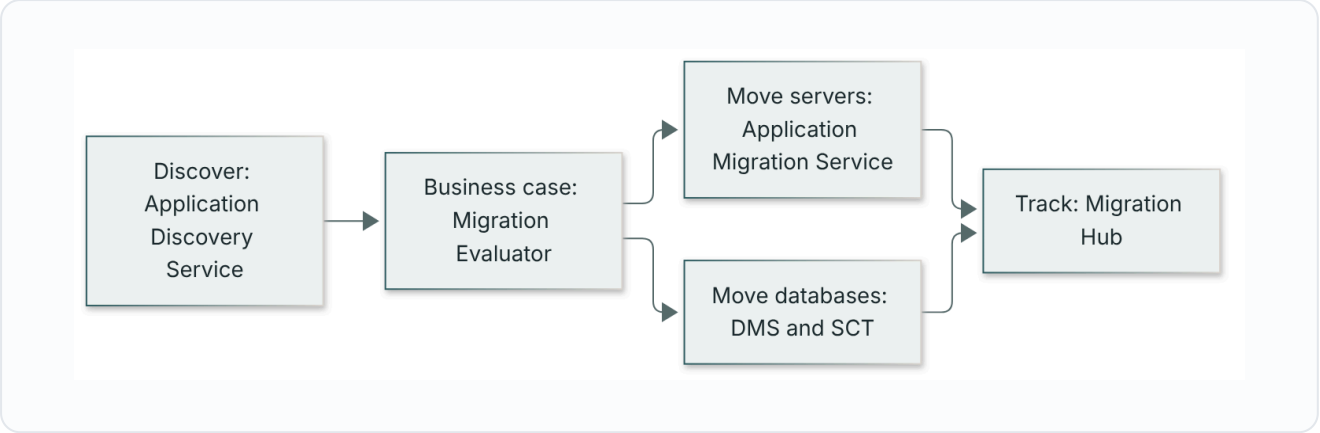
Repurchase, "drop and shop". Abandon the existing application and **buy a different product**, usually SaaS. Example: replace an old in-house customer management tool with a SaaS CRM.

Refactor or re-architect. Redesign and rewrite the application to use cloud-native features (serverless, managed services, microservices). It takes the **most time and effort** but gives the **most benefits**: scalability, agility, lower long-term costs.

Strategy	Effort	When it fits	Example
Retire	None	Nobody uses it	Old reporting tool
Retain	None	Must stay or not ready	System with legal constraints
Rehost	Low	Move fast, no changes	Servers copied to EC2
Relocate	Low	Move a whole platform	VMware to VMware Cloud on AWS
Replatform	Medium	Small gains without rewriting	Database moved to RDS
Repurchase	Medium	A better product exists	In-house CRM to SaaS
Refactor	High	Maximum cloud benefits	Monolith to serverless

The migration tools

A migration usually follows the same path: **discover** what you have, **build a business case**, **move** servers and databases, **track** progress.



- **AWS Application Discovery Service**: collects information about the servers in your data center (configuration, usage, which server talks to which) so you know what you have before planning.
- **Migration Evaluator** NEW (formerly TSO Logic): analyzes your current usage and estimates what it would cost on AWS, to build a **business case** for managers.
- **AWS Application Migration Service**: the main tool to **rehost**. It continuously copies your servers to AWS while they keep running, then lets you switch over with very little downtime.
- **AWS Database Migration Service (DMS)**: moves databases to AWS. The source database **keeps working during the copy**, and DMS keeps copying new changes (**database replication**) until you switch. Result: minimal downtime. It works between identical engines (MySQL to MySQL) and different ones (Oracle to Aurora).
- **AWS Schema Conversion Tool (SCT)**: when you change database engine, it converts the **schema** (the structure of tables, views, procedures) from one engine to the other. SCT converts the structure, DMS moves the data.
- **AWS Migration Hub**: a single dashboard to follow the progress of all migrations.
- **AWS DataSync**: an **online** transfer service. A small agent installed next to your storage copies files from on-premises file servers (NFS, SMB), Hadoop or other object storage into **S3, EFS or FSx**, quickly and securely (encryption, integrity checks), once or on a schedule. Use it to **move or regularly synchronize large amounts of files** over the network.
- **AWS Transfer Family** OUT: managed **SFTP, FTPS and FTP** servers that store files directly in S3 or EFS, for partners who exchange files with these protocols.
- **AWS Snow Family** LEGACY: rugged physical devices that AWS ships to you; you copy your data onto them and ship them back, and AWS loads the data into S3. Used when the network is too slow for terabytes or petabytes (for example, 100 TB over a 100 Mbps line would take months). **Snowball Edge** (storage optimized or compute optimized, can also run EC2 instances at remote sites), **Snowcone** (small, portable) and **Snowmobile** (a truck for exabytes). As with any transfer into AWS, importing the data into S3 is not charged per GB; you pay for the device job and extra days. Snowmobile and Snowcone have been retired, and Snowball Edge is no longer offered to new customers: AWS now points to DataSync

(with Direct Connect), the AWS Data Transfer Terminal (an AWS site where you bring your own disks) and AWS Outposts for edge computing.

- **AWS Mainframe Modernization** OUT: tools to migrate and modernize **mainframe** applications (COBOL) to AWS.
- AWS Launch Wizard LEGACY: a guided way to deploy third-party applications such as Microsoft SQL Server or SAP on AWS following best practices.

> ON THE EXAM
> "Fastest migration without changing the application" = Rehost. "Move the database to a managed service with small changes" = Replatform. "Replace with a SaaS product" = Repurchase. "Rewrite to be cloud-native" = Refactor. "Estimate costs to build a business case" = Migration Evaluator. "Migrate a database with minimal downtime" = DMS; "convert from Oracle to PostgreSQL" = SCT + DMS.

1.4 Cloud economics

Fixed costs and variable costs

IN PLAIN WORDS

A **fixed cost** stays the same whatever you do; a **variable cost** goes up and down with your activity.

On-premises IT is mostly **fixed**. Sara bought servers for 10,000 euros: whether she sells 10 cakes or 10,000, the money is spent. Such purchases are **CapEx** (capital expenditure): a big investment upfront in assets that lose value over years (depreciation).

Cloud IT is mostly **variable**. Sara pays, say, 40 euros in a quiet month and 120 euros in December. This is **OpEx** (operational expenditure): a running cost, paid as you go.

Why does it matter? Variable costs mean **no big upfront investment**, **less financial risk** (if a project fails, you stop paying), and costs that **follow revenue**.

The real cost of on-premises

When comparing on-premises and cloud, people often compare only the price of a server with the price of a cloud instance. That is misleading. The full cost of owning infrastructure, called **total cost of ownership (TCO)**, includes many hidden costs:

- **Hardware** (servers, storage, network equipment), replaced every 3 to 5 years.
- **Data center space**: rent or construction, racks, physical security.
- **Electricity and cooling**: servers run day and night and produce heat.
- **Staff**: people to install, maintain, repair and secure the hardware.
- **Software licenses and maintenance contracts**.

- **Spare capacity** bought "just in case" and rarely used.
- **Time**: weeks to get new hardware, which slows the business.

Licenses: BYOL or license included

What a license is. Commercial software such as Microsoft Windows Server, SQL Server or Oracle Database is not free: you pay the vendor for the right to use it. That right is a **license**. Licenses are often counted per server, per CPU socket or per CPU core.

On AWS you have two options:

- **License included**: the price of the AWS resource **already contains** the license. For example, a Windows EC2 instance costs a bit more per hour than a Linux one because the Windows license is included. Simple: nothing to buy or track.
- **BYOL, bring your own license**: you **reuse licenses you already own** from your on-premises servers, so you do not pay twice. Some licenses are tied to physical hardware (sockets or cores), which is why AWS offers **Dedicated Hosts**: a whole physical server for you, where you can see the sockets and cores. **AWS License Manager** helps you track how many licenses you use and stay within the vendor's rules.

Rightsizing

Rightsizing means choosing the resource type and size that **match the real need**. Many servers are oversized "to be safe". If a server uses only 10% of its CPU, you pay for 90% of nothing: switch to a smaller, cheaper size. Rightsizing is an ongoing habit, because needs change. AWS helps with **AWS Compute Optimizer** and the recommendations in **AWS Cost Explorer**, which analyze your actual usage and suggest better sizes.

Automation

Automation means letting scripts, templates and services do the work humans used to do by hand. Benefits: **fewer human errors**, **faster** deployments, **less staff time**, the **same result every time**, and resources that run **only when needed** (for example, test environments created in the morning and deleted in the evening). Tools you will meet: **AWS CloudFormation** (infrastructure as code) and **Auto Scaling**.

Economies of scale

Because AWS serves millions of customers, it buys hardware and energy in enormous quantities, builds its own optimized hardware, and spreads its costs over everyone. Its cost per unit is far lower than any single company could achieve, and AWS passes part of these savings to customers through lower prices.

ON THE EXAM

"Avoid large upfront investments" = variable expense, OpEx. "Costs of on-premises" = hardware, space, power, cooling, staff, licenses. "Reuse existing per-core licenses" = BYOL + Dedicated Hosts, tracked with License Manager. "Server only 10% used" = rightsizing. "Lower prices thanks to AWS's size" = economies of scale.

KEY TAKEAWAYS

- Cloud computing = **on-demand delivery of IT resources over the internet with pay-as-you-go pricing**. It removes the purchase, the data center and the guessing.
- Virtualization lets AWS split physical servers into isolated virtual machines for many customers.
- IaaS (EC2: you manage the OS), PaaS (Elastic Beanstalk, RDS: you bring code and data), SaaS (a finished application).
- Deployment models: cloud, hybrid (connected with Direct Connect, VPN, Storage Gateway, Outposts), on-premises.
- The six advantages: variable expense, economies of scale, stop guessing capacity, speed and agility, no data center work, go global in minutes.
- High availability = no single point of failure (several AZs). Elasticity = automatic scaling up and down (Auto Scaling). Durability ≠ availability.
- Six Well-Architected pillars, each with its question: operations, security, reliability, performance, cost, sustainability.
- CAF prepares the organization: six perspectives, four benefits. 7 Rs decide each application's fate; DMS, SCT, Application Migration Service do the move.
- Economics: fixed → variable, TCO hidden costs, BYOL vs license included, rightsizing, automation, economies of scale.

PART 2 · DOMAIN 2

Security and Compliance

30% of the scored content, the second biggest domain

Who protects what in the cloud, how to control who can access your resources, which services protect and watch your workloads, and how to prove that you follow the rules. Most of it is logic: once you understand the shared responsibility model, many questions answer themselves.

IN THIS PART

2.1 Understand the AWS shared responsibility model

2.2 Understand AWS Cloud security, governance, and compliance concepts

2.3 Identify AWS access management capabilities

2.4 Identify components and resources for security

OUR STORY THROUGH THIS COURSE

Sara's shop is online and orders are coming in. Now she stores customer names, addresses and order histories. She must protect them: from hackers, from mistakes, and from people who should not see them. And her bank asks her to prove that payments are handled securely.

2.1 The shared responsibility model

Why security is "shared"

IN PLAIN WORDS

On-premises, the company is responsible for **everything**: the building's locks, the servers, the network, the software, the data. When it moves to AWS, part of this work is done by AWS, and part stays with the customer. The **shared responsibility model** is the official description of **who does what**. It is one of the most tested topics on the exam.

AWS sums it up in two short sentences, worth learning by heart:

- **AWS is responsible for security OF the cloud**: the infrastructure that runs all the services.
- **The customer is responsible for security IN the cloud**: everything the customer puts in the cloud and how they configure it.

ANALOGY: RENTING AN APARTMENT

The landlord is responsible for the building: the walls, the roof, the main entrance door with its code, the stairs, the electricity network. You, the tenant, are responsible for your apartment: locking your own door, choosing who gets a copy of your key, closing your windows, and protecting what you keep inside. If you leave your door open and get robbed, it is not the landlord's fault.

What AWS takes care of: security OF the cloud

- **Physical security of data centers**: guarded buildings, controlled access with badges and biometrics, cameras, staff background checks. Customers cannot even visit them. Old disks are destroyed securely.
- **Hardware**: servers, storage devices, network equipment, and their maintenance and replacement.
- **The global network** between data centers and Regions.
- **The virtualization layer** (hypervisor) that keeps customers isolated from each other.
- **The software of managed services**: for example the database engine of Amazon RDS or the servers behind AWS Lambda, including their patching.

What the customer takes care of: security IN the cloud

- **Customer data**: always the customer's responsibility, whatever the service. You decide what you store, who can read it and whether it is encrypted.
- **Identity and access management**: who has an account, which permissions, whether MFA is enabled (section 2.3).
- **Operating system, applications and their patches** when you use infrastructure services like EC2.
- **Network and firewall configuration**: security groups, network access control lists (section 3.5).
- **Encryption choices**: turning encryption on, managing keys when you choose to.

Customer: security IN the cloud	AWS: security OF the cloud
Customer data	Software of the managed services (compute, storage, database, networking)
Identity and access management	Hardware and global infrastructure: Regions, Availability Zones, edge locations
Guest operating system, applications, patches (on EC2)	Virtualization layer
Network and firewall configuration, encryption options	Physical security of the data centers

The line moves with the service

The more **managed** a service is, the more AWS does, and the less is left to you. Let us follow Sara with three ways of running her database.

Option 1, database on Amazon EC2 (IaaS). Sara rents a virtual server and installs MySQL herself. AWS protects the data center, the hardware and the hypervisor. Sara must patch the **operating system**, install and patch **MySQL**, configure backups, set the **firewall** (security group), manage database users, and protect the data.

Option 2, Amazon RDS (managed database). AWS also handles the operating system, the **installation and patching of the database engine**, and automated backups. Sara still manages the **data**, the **database users and passwords**, **who can connect** (network settings), and whether encryption is enabled.

Option 3, AWS Lambda for her order-processing code (serverless). AWS manages the servers, operating system, runtime and scaling. Sara is responsible for her **code**, its **permissions**, and the **data** it handles.

Service	AWS takes care of	The customer takes care of
Amazon EC2	Data center, hardware, hypervisor	**Guest OS and its patches**, applications, security groups, data, accounts
Amazon RDS	Plus the OS, **database engine installation and patching**, automated backups	Data, database users, network access, encryption settings
AWS Lambda	Plus servers, OS, runtime, scaling	Code, function permissions, data
Amazon S3	Infrastructure, durability, availability	Who can access (bucket policies, IAM), encryption choice, the data

Shared controls

Some security controls apply to **both** parties, each in its own layer:

- **Patch management**: AWS patches the infrastructure and managed services; the customer patches their guest operating systems and applications.
- **Configuration management**: AWS configures its infrastructure devices; the customer configures their operating systems, databases and applications.
- **Awareness and training**: AWS trains its employees; the customer trains its own.

> ON THE EXAM
>
> "Who patches the guest OS of an EC2 instance?" The customer. "Who patches the database engine on RDS?" AWS. "Who is responsible for the physical security of data centers?" AWS. "Who configures security groups?" The customer. "Who is responsible for customer data?" **Always the customer.** "Patch management" is a **shared** control.

2.2 Security, governance and compliance concepts

Three words: security, governance, compliance

- **Security** protects data and systems against attacks, mistakes and leaks.
- **Governance** is the set of rules a company sets for itself and the means to enforce them: who may create what, which Regions are allowed, how much can be spent.
- **Compliance** means **following external rules**: laws and industry standards, and being able to **prove** it to an auditor.

GOOD TO KNOW

Security is often summarized by the **CIA triad**. **Confidentiality**: only authorized people can see the data (access control, encryption). **Integrity**: the data is not altered without permission (logs, checksums). **Availability**: the data and services are there when needed (redundancy, DDoS protection). Every security service protects one or more of these three.

Compliance: proving that you follow the rules

IN PLAIN WORDS

Many activities are regulated. Before a bank lets Sara accept card payments, she must respect card security rules. A hospital must protect health records. A European company must protect personal data of EU citizens. Each set of rules is called a **compliance program** or **standard**, and companies are checked by **auditors**.

The main ones you should recognize:

Program	What it is about
PCI DSS	Security of **payment card** data
HIPAA	Protection of **health data** in the United States
GDPR	Protection of **personal data** of people in the European Union
SOC 1, SOC 2, SOC 3	Independent audit reports on a company's controls (financial, security, availability)
ISO 27001	International standard for managing information security
FedRAMP	Security requirements for cloud services used by the US government

How it works with AWS. AWS has its infrastructure and many services audited and certified against these programs. That covers **AWS's part**. But because of the shared responsibility model, **using a certified service does not make your application compliant automatically**: you must still configure and use the services correctly (encryption, access control, logging). Also:

- Requirements **differ by country and by industry**: a German hospital, a US bank and a French retailer have different obligations.
- **Not every AWS service is covered by every program**: before storing health data in a service, check that the service is in scope of HIPAA on AWS's compliance pages.

AWS Artifact is the place to **get the proof**. It is a free, self-service portal in the AWS console with two parts:

- **Artifact Reports**: download AWS's audit reports and certifications (SOC reports, PCI DSS attestation, ISO certificates) to give to your auditors.
- **Artifact Agreements**: review and accept legal agreements with AWS, such as the **Business Associate Addendum (BAA)** needed for HIPAA.

> EXAMPLE
> Sara's bank asks for proof that the infrastructure hosting payment data is PCI DSS compliant. Sara opens AWS Artifact, downloads the AWS PCI DSS attestation and sends it. For her own part, she shows that she encrypts data and restricts access.

Encryption

IN PLAIN WORDS

Encryption transforms readable data into unreadable characters using a secret value called a **key**. Only someone with the right key can turn it back into readable data (**decryption**). If a thief steals encrypted data without the key, it is useless to them.

There are two moments when data needs protection:

- **In transit**: while it travels over a network, for example between a customer's browser and Sara's website. Protected with **TLS** (the protocol behind **HTTPS** and the padlock in your browser). TLS needs a **certificate** that proves the website's identity.
- **At rest**: while it is stored, on a disk, in a database, in S3. Protected by encrypting the storage with a key.

The AWS services:

Service	What it does in plain words	When to use it
AWS Key Management Service (KMS)	Creates, stores and controls **encryption keys** for you, and lets you decide who may use them. Integrated with most AWS services: encrypting an EBS disk, an S3 bucket or an RDS database is often a single checkbox	The default choice for **encryption at rest**
AWS CloudHSM	Gives you a **dedicated hardware security module** (HSM): a physical, tamper-resistant device used only by you, in which **you alone** control the keys	Strict regulations that require single-tenant hardware and full customer control of keys

Service	What it does in plain words	When to use it
AWS Certificate Manager (ACM)	Provides and **renews automatically** the SSL/TLS certificates for your websites; public certificates are free	**Encryption in transit** (HTTPS)
AWS Secrets Manager	Stores passwords and API keys securely and can rotate them	Secrets (see 2.3)

NOTE

To encrypt data **inside your application** before sending it anywhere (client-side encryption), AWS provides the **AWS Encryption SDK**, a free library that works with KMS keys.

NOTE

Encryption is one of the **benefits of cloud security** in the official objective: on AWS it is easy, cheap and integrated, while doing it on-premises often required expensive equipment and expertise.

ON THE EXAM

"Encrypt data at rest with managed keys" = **KMS**. "Dedicated hardware, customer-only control of keys" = **CloudHSM**. "SSL/TLS certificate for HTTPS" = **ACM**. "Data in transit" = TLS. "Download a SOC 2 or PCI report" = **Artifact**. "Is the customer automatically compliant because AWS is?" **No.**

Logs, monitoring and governance

IN PLAIN WORDS

A **log** is a record of events: who connected, what was changed, which error occurred. Logs are essential to investigate incidents and to prove compliance. Three AWS services record different things, and the exam loves to mix them up.

The question	The service	What it records
WHO did **WHAT** and **WHEN**?	**AWS CloudTrail**	Every **API call** made in your account: user, action, time, source IP address
HOW is it performing **NOW**?	**Amazon CloudWatch**	**Metrics** (CPU, traffic, errors), **logs** from applications, **alarms**, dashboards
WHAT is the configuration, and is it **COMPLIANT**?	**AWS Config**	The **configuration history** of each resource and its compliance with your rules

AWS CloudTrail. Every action in AWS, whether a click in the console, a CLI command or an application call, is an **API call**. CloudTrail records them as **events**. The last 90 days of management events are visible for free in the event history, and you can create a **trail** to keep logs longer in S3. It is your **audit trail**: the answer to "who deleted this database?".

Amazon CloudWatch. The monitoring service. It collects **metrics** (numbers over time, like CPU usage or number of requests), stores **logs** sent by applications and services (**CloudWatch**

Logs), displays **dashboards**, and triggers **alarms** when a metric crosses a threshold: send an email, add servers, restart an instance. A **billing alarm** in CloudWatch warns you when your estimated bill passes an amount.

AWS Config. Records the **configuration** of your resources over time (for example: "this S3 bucket was private until Tuesday, public since then") and evaluates them against **rules** you choose ("all disks must be encrypted", "no bucket may be public"). Non-compliant resources are flagged, and you can fix them automatically.

> EXAMPLE
> One morning, one of Sara's storage buckets has become public. **CloudTrail** shows that a developer called `PutBucketPolicy` at 14:02 from a given IP address. **AWS Config** shows the bucket's configuration before and after the change and marks it **non-compliant** with the rule "no public buckets". **CloudWatch** shows a spike of downloads and had triggered an **alarm** that emailed Sara.

Governance services:

- **AWS Organizations**: groups many AWS accounts under one **management account**, with one bill. **Service control policies (SCPs)** set the **maximum** permissions any account in the organization can have (guardrails), for example "nobody may use Regions outside Europe" or "nobody may turn off CloudTrail".
- **AWS Control Tower**: sets up a secure, well-organized multi-account environment (a **landing zone**) automatically, with recommended guardrails already in place.
- **AWS Systems Manager**: operates your servers at scale (patching, running commands, inventory) and stores configuration values.
- **Access reports**: IAM produces the **credential report** (every user with the status of their passwords, keys and MFA) and **last-accessed information** (which permissions are really used). Auditors love them.

> NOTE
> The services that help customers secure their resources, such as Inspector, Security Hub, GuardDuty and Shield, are explained in section 2.4.

> ON THE EXAM
> "Audit who made an API call" = CloudTrail. "Alarm when CPU passes 80%", "billing alarm" = CloudWatch. "Track configuration changes and check compliance with rules" = AWS Config. "Prevent all accounts from using a service" = SCP in AWS Organizations. "Report of all users and their credentials" = credential report.

2.3 Access management

Authentication and authorization

Every access control system answers two questions:

- **Authentication**: **who are you?** You prove your identity with a password, a code, a key.
- **Authorization**: **what are you allowed to do?** Once identified, your permissions decide which actions you may perform on which resources.

> ANALOGY: AN OFFICE BUILDING
> At the entrance you show your badge: that is authentication. The badge then opens some doors and not others: that is authorization.

The root user

When you create an AWS account, you sign up with an email address and a password. This identity is the **root user**. It has **complete, unrestricted access** to everything in the account, including the billing, and the power to close the account. Nothing can limit it.

That makes it dangerous: if someone steals the root credentials, they own your account. So AWS recommends:

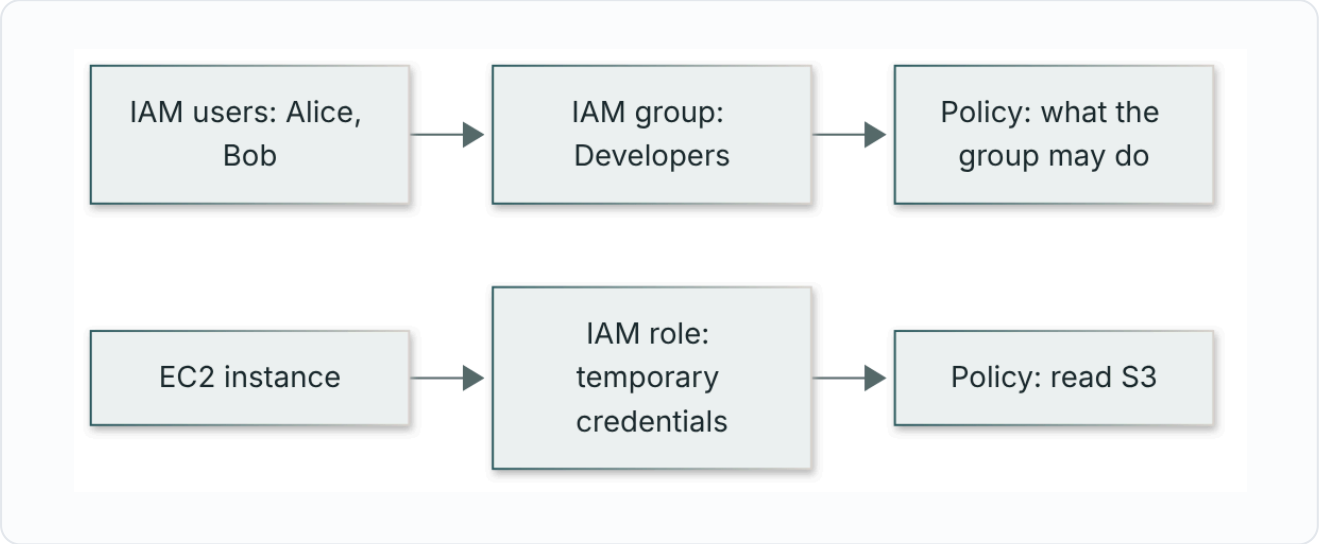
- Enable **MFA** on the root user immediately.
- Use a **strong, unique password**.
- **Never create access keys** for the root user.
- **Do not use it for daily work**: create other identities with only the permissions they need, and lock the root credentials away.
- Use it only for the few **tasks that only the root user can perform**:
  - change account settings: account name, root email address, root password;
  - **close the AWS account**;
  - **change or cancel the AWS Support plan**;
  - restore IAM permissions when every administrator has locked themselves out;
  - register as a seller in the **Reserved Instance Marketplace**;
  - enable **MFA Delete** on an S3 bucket;
  - sign up for AWS GovCloud (US).

> ANALOGY
> The root user is the master key of a building. You keep it in a safe and give employees personal badges.

AWS Identity and Access Management (IAM)

IN PLAIN WORDS

IAM is the free AWS service that lets you create identities and control **who can do what on which resources** in your account. It is a **global** service (not tied to a Region).



IAM users. An identity for **one person or one application**, with **long-term credentials**: a password to sign in to the console and/or access keys for programs. Sara creates a user for herself and one for her developer, so nobody shares a login.

IAM groups. A collection of users. You attach permissions to the group once, and every member gets them. When someone joins the team, add them to the group; when they leave, remove them. A user can be in several groups. Groups cannot contain other groups.

IAM roles. An identity that **has no permanent password or keys**. Instead, someone or something **assumes** the role and receives **temporary credentials** that expire automatically (from minutes to hours). This is safer: nothing permanent to steal. Roles are used for:

- **AWS services**: an EC2 instance that needs to read files in S3 gets a role, instead of storing keys on the server.
- **Cross-account access**: a user from another AWS account temporarily works in yours.
- **Federated users**: employees who sign in with their company account (see below).

IAM policies. A **policy** is a document, written in **JSON**, that lists permissions. You attach it to a user, a group or a role. Example:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::sara-invoices/*"
  }]
}
```

Read it like a sentence: **Effect** (Allow or Deny) **Action** (here, read a file from S3) on **Resource** (every file in the bucket `sara-invoices`). An optional **Condition** can add rules such as "only from this IP address" or "only with MFA".

There are two kinds of policies:

- **Managed policies**: standalone policies you can attach to many identities. **AWS managed** policies are written and maintained by AWS for common jobs (for example `AmazonS3ReadOnlyAccess`); **customer managed** policies are written by you and reusable.
- **Inline (custom) policies**: written directly inside one user, group or role, for a single special case.

How AWS decides. When someone tries an action, AWS evaluates all policies that apply:

1. By default, everything is **denied** (implicit deny).
2. If a policy **allows** the action, it is permitted...
3. ...unless any policy explicitly **denies** it: an **explicit deny always wins**.

The principle of least privilege

Give every identity only the permissions it needs to do its job, and nothing more. Sara's developer needs to deploy the website, not to read the accounting data or close the account. If his credentials are stolen, the damage stays limited.

How to apply it: start with minimal permissions and add when needed, use groups and roles rather than many individual permissions, review regularly. Tools: **IAM Access Analyzer** (finds resources shared with outside accounts and helps write tight policies), **last-accessed information** (shows permissions never used, to remove them), and the **credential report**.

Proving who you are: credentials and authentication methods

Passwords and password policies. IAM users who use the console have a password. A **password policy** sets account-wide rules: minimum length, required character types, expiration, no reuse of old passwords.

Multi-factor authentication (MFA). A password alone can be stolen or guessed. MFA adds a second proof: **something you know** (the password) plus **something you have** (a device). Types supported by AWS:

- **Virtual MFA**: an authenticator app on a phone that shows a six-digit code changing every 30 seconds.
- **Hardware TOTP token**: a small physical device that displays codes.
- **FIDO security keys and passkeys**: a key you plug in or touch (or your phone's biometrics).

Enable MFA on the root user and on every user with significant permissions.

Access keys. Programs (the CLI, SDKs, scripts) cannot type a password in a web page. They use **access keys**: an **access key ID** (like a username) and a **secret access key** (like a password). Rules: never share them, **never put them in code or push them to GitHub** (bots scan for them within minutes), rotate them regularly, delete unused ones, never create them for root, and **prefer IAM roles** wherever possible.

Storing secrets. Applications need secrets: database passwords, API keys of payment providers. Never write them in the code. Use:

- **AWS Secrets Manager**: stores secrets encrypted, controls who can read them, and can **rotate them automatically** (for example change the database password every 30 days without breaking the application).
- **AWS Systems Manager Parameter Store**: stores configuration values and secrets, simpler and cheaper, without built-in automatic rotation for most cases.

Identities for employees and for customers

IAM Identity Center (formerly AWS Single Sign-On). Large companies have many AWS accounts and many business applications. Creating an IAM user in each account for each employee would be unmanageable. IAM Identity Center provides **single sign-on (SSO)**: an employee signs in **once**, with their company identity, and accesses **all the AWS accounts and applications** they are allowed to use. It connects to the company directory (Microsoft Active Directory, Microsoft Entra ID, Okta) or has its own.

Federation. "Federated identity" means trusting identities managed by **another system** instead of creating new users in AWS. The company's **identity provider (IdP)** authenticates the employee, and AWS gives them a role with temporary credentials. Standards used: **SAML 2.0** and **OpenID Connect (OIDC)**.

Cross-account IAM roles. A user in account A assumes a role in account B to work there temporarily, without a second user.

Amazon Cognito. For the **customers of your application**, not your employees. Cognito handles sign-up, sign-in and password recovery for millions of app users, including "Sign in with Google, Apple or Facebook". Sara uses it for customer accounts on her shop.

AWS Directory Service. Runs a managed **Microsoft Active Directory** in AWS, for applications that need one.

Who signs in	Service
Your employees, across many AWS accounts and apps	**IAM Identity Center** (with federation)
An application or AWS service (EC2, Lambda)	**IAM role**
A user from another AWS account	**Cross-account IAM role**
The customers of your web or mobile app	**Amazon Cognito**

Who signs in	Service
A small team in one account	IAM users in groups

AWS Security Token Service (STS). The service behind every role: it issues the **temporary credentials** (valid from minutes to hours) that a user, an application or another account receives when it **assumes a role** or signs in through federation.

Connecting to servers securely

Administrators sometimes need a command line on an EC2 instance. Two modern options avoid long-lived SSH keys and open ports:

- **EC2 Instance Connect**: connect by SSH with a **temporary key** pushed through IAM, so access is controlled by IAM policies and logged in CloudTrail.
- **AWS Systems Manager Session Manager**: open a shell from the browser or the CLI **without opening any inbound port**, without SSH keys and without a bastion host; every session can be recorded. It relies on the **SSM Agent** installed on the instance (preinstalled on Amazon Linux and many AMIs).

First steps in a new account

1. Turn on **MFA** for the root user.
2. **Never** create access keys for root.
3. Create an **administrator identity** (an IAM user in an admin group, or IAM Identity Center) and use it for daily work.
4. Create a **budget** with an email alert (section 4.2), so a forgotten resource never surprises you.

> GOOD TO KNOW
> In a new account, the classic first ten minutes: an IAM user in an *admin* group, an **account alias** so the sign-in address is easy to remember, and MFA with a phone app or a hardware key. Do it in your own Free Tier account.

Zero trust. A modern security idea: **never trust automatically** because a request comes from "inside" the network; **always verify** identity and permissions for each request. In the cloud, **identity is the main security perimeter**, which is why IAM, MFA and least privilege matter so much.

ON THE EXAM

"Give an EC2 instance access to S3" = **IAM role** (never keys on the server). "Employees need one login for many AWS accounts" = **IAM Identity Center**. "Use the company's existing identities" = **federation**. "Sign-in for the users of a mobile app" = **Cognito**. "Rotate database passwords automatically" = **Secrets Manager**. "Permissions for a team" = **group**. "Grant only the needed permissions" = **least privilege**. "Close the account" or "change the Support plan" = **root user**.

2.4 Security components and resources

IN PLAIN WORDS

AWS offers a security service for each job: blocking attacks, detecting threats, finding weaknesses, finding sensitive data, investigating, and giving one overall view. The easiest way to remember them is to follow an attack on Sara's shop.



Blocking attacks at the door

AWS Shield: protection against DDoS. A **DDoS attack** (distributed denial of service) floods a website with so much fake traffic, from thousands of infected machines, that real customers cannot get in. Shield absorbs and filters these floods.

- **Shield Standard**: **free and automatic** for every AWS customer, protects against the most common network attacks.
- **Shield Advanced**: **paid**, for critical applications. Adds protection against larger and more sophisticated attacks, a 24/7 **Shield Response Team** (SRT, formerly called the DDoS Response Team) of AWS experts, detailed reports, and **cost protection** (AWS credits you for the extra scaling costs caused by an attack).

AWS WAF: web application firewall. Some attacks do not flood the site; they send cleverly crafted requests to trick the application. For example **SQL injection** (inserting database commands in a form field to steal data) and **cross-site scripting (XSS)** (injecting malicious scripts into pages). WAF inspects each web request (it works at the application layer, "layer 7") and blocks those that match your rules or AWS managed rules, as well as known bad IP addresses, bots, or too many requests from one source. It protects CloudFront, load balancers and API Gateway.

AWS Firewall Manager. A company with fifty AWS accounts needs the same WAF rules, Shield Advanced protections and security group rules everywhere. Firewall Manager applies and enforces them **centrally across all accounts** of an AWS Organization.

Detecting threats and weaknesses

Amazon GuardDuty: threat detection. GuardDuty continuously analyzes your account's activity, using **CloudTrail logs, VPC Flow Logs (network traffic) and DNS logs**, with machine learning and threat intelligence. It raises **findings** when something looks wrong: an instance talking to a known malicious server, crypto-mining, unusual API calls from an unknown country, credentials used in a strange way. You turn it on with one click; there is nothing to install.

Amazon Inspector: vulnerability scanning. Inspector automatically scans your **EC2 instances, container images and Lambda functions** for **known software vulnerabilities** (CVEs, missing patches) and unintended network exposure (open ports), and gives a risk score so you fix the worst first.

Amazon Macie: sensitive data in S3. Macie uses machine learning to discover **sensitive data**, such as names, addresses, credit card numbers or ID numbers (**personally identifiable information, PII**), stored in **S3 buckets**, and alerts you if such data is exposed or unencrypted. Useful for GDPR.

Amazon Detective: investigation. When GuardDuty raises a finding, you need to understand what really happened. Detective collects logs and builds visual graphs of the activity over time to help you find the **root cause** quickly.

AWS Security Hub: one dashboard. Instead of checking GuardDuty, Inspector, Macie and others separately, Security Hub **gathers all findings in one place**, prioritizes them, and runs automated checks of your accounts against security best practices and standards. This is called **cloud security posture management (CSPM)**.

AWS Trusted Advisor: best-practice checks. Trusted Advisor inspects your account and gives recommendations, including **security** ones: MFA not enabled on root, security groups open to the whole internet on sensitive ports, public S3 buckets, exposed access keys. (It also covers cost, performance, fault tolerance and limits, see 4.3.)

Service	Job	Key words in a question
Shield	Stop **DDoS** floods (Standard free, Advanced paid)	DDoS
WAF	Block **web exploits** (SQL injection, XSS), bad IPs, bots	SQL injection, layer 7, web requests
Firewall Manager	Manage firewall rules **across accounts**	central, organization-wide rules
GuardDuty	**Detect threats** from logs	malicious activity, compromised, unusual behaviour
Inspector	**Scan for vulnerabilities**	CVE, patches, software flaws
Macie	Find **sensitive data in S3**	PII, personal data, GDPR

Service	Job	Key words in a question
Detective	**Investigate** the root cause	investigate, analyze a finding
Security Hub	**One view** of all findings and best-practice checks	single dashboard, aggregate
Trusted Advisor	Best-practice **recommendations** on the account	recommendations, checks
AWS Network Firewall OUT	Network firewall for a VPC	

GOOD TO KNOW

A useful way to separate two "checking" services: **Inspector** examines your **workloads** (an instance, a container image) for software vulnerabilities. **Trusted Advisor** examines your **account** for best practices across cost, performance, security, fault tolerance and limits. Both run checks, but on different things.

Defense in depth. Never rely on a single protection. Secure **every layer**: data (encryption), application (WAF, secure code), compute (patching, Inspector), network (security groups, network ACLs), perimeter (Shield, CloudFront), identity and access (IAM, MFA), and the physical layer (AWS's job). If one layer fails, the next one still protects you.

Third-party security products. If a company already trusts a security vendor (firewall, antivirus, monitoring), it can usually find and buy that product in **AWS Marketplace** and deploy it in a few clicks, billed on the AWS bill.

Where to find security information and help

Resource	What you find there
AWS Security Center	AWS's security overview, best practices and compliance resources
AWS Security Blog	Articles and news on AWS security features and practices
AWS Knowledge Center	Answers to the most frequent questions received by AWS Support
AWS documentation, whitepapers	Detailed official guidance, such as the security pillar of Well-Architected
AWS re:Post	Community questions and answers
AWS Trust & Safety team	Where to **report abuse** coming from AWS resources: spam, attacks, malware hosted on AWS

Penetration testing. A penetration test is a simulated attack on your own systems to find weaknesses. AWS allows customers to test their own resources on a list of common services (EC2, RDS, Lambda, and others) **without asking for permission first**. Simulating denial-of-service attacks follows a separate, stricter policy.

ON THE EXAM

DDoS = **Shield**. SQL injection, XSS = **WAF**. Same rules in all accounts = **Firewall Manager**. Threat detection = **GuardDuty**. Vulnerability scan = **Inspector**. PII in S3 = **Macie**. Root cause = **Detective**. Single pane = **Security Hub**. Account best-practice checks = **Trusted Advisor**. Buy a third-party firewall = **Marketplace**. Report abuse = **Trust & Safety**.

KEY TAKEAWAYS

- AWS secures the cloud (facilities, hardware, network, hypervisor, managed service software). You secure what you put in it (data, IAM, configuration, guest OS on EC2).
- The more managed the service (EC2 → RDS → Lambda), the more AWS takes over. Data is always yours. Patch and configuration management are shared.
- Compliance programs (PCI DSS, HIPAA, GDPR, SOC, ISO) vary by industry, country and service. AWS Artifact gives AWS's reports and agreements; AWS being compliant does not make you compliant.
- Encryption in transit (TLS, ACM) and at rest (KMS; CloudHSM for dedicated hardware).
- CloudTrail = who did what. CloudWatch = metrics, logs, alarms. Config = configuration history and compliance. Organizations + SCPs = guardrails.
- Root user: MFA, no access keys, only for root-only tasks. IAM: users, groups, roles (temporary credentials), JSON policies; explicit deny wins; least privilege.
- MFA, password policies, access keys (never in code), Secrets Manager (rotation). Identity Center = employees' SSO; Cognito = app customers; federation = existing identities.
- Shield, WAF, Firewall Manager, GuardDuty, Inspector, Macie, Detective, Security Hub, Trusted Advisor: one job each.

PART 3 · DOMAIN 3

Cloud Technology and Services

34% of the scored content, the largest domain

The exam describes a need and asks which service fits. For each service, remember **what problem it solves** and **when to choose it**, not how to configure it. This part follows the eight task statements of the guide.

IN THIS PART

3.1 Define methods of deploying and operating in the AWS Cloud

3.2 Define the AWS global infrastructure

3.3 Identify AWS compute services

3.4 Identify AWS database services

3.5 Identify AWS network services

3.6 Identify AWS storage services

3.7 Identify AWS AI/ML services and analytics services

3.8 Identify services from other in-scope AWS service categories

OUR STORY THROUGH THIS COURSE

Sara's shop is growing. She needs servers that cope with peaks, a database for orders, storage for thousands of cake photos, fast pages for customers abroad, and maybe a chatbot. In this part, we build her whole architecture service by service.

3.1 Deploying and operating in the AWS Cloud

Everything in AWS is an API

IN PLAIN WORDS

An **API** (application programming interface) is a door through which programs talk to a service. Every action in AWS, "create a server", "upload a file", "delete a database", is a request sent to an AWS **API**. All the tools below are just different ways of sending these requests: by clicking, by typing commands, by writing code or by describing what you want in a file.

Provisioning means creating and preparing resources (servers, networks, databases) so they are ready to use. **Operating** means running and managing them afterwards.

The ways to access AWS

The AWS Management Console. The website where you sign in and **click**. Every service has its page with buttons and forms. Ideal for **learning**, exploring, looking at dashboards and doing **one-time** tasks. Not ideal for repeating the same setup fifty times: it is slow and people make mistakes. There is also a mobile app for checking resources and alarms.

The AWS CLI (command line interface). A program you install on your computer to manage AWS by **typing commands** in a terminal, for example `aws s3 ls` to list your storage buckets. Commands can be put in **scripts** to automate repetitive tasks.

The AWS SDKs (software development kits). Libraries for programming languages (Python, Java, JavaScript, .NET, Go...) so that **your application's code** can call AWS. Sara's shop uses the SDK to store each new order photo in S3.

AWS Tools for PowerShell and IDE toolkits. AWS commands for Windows PowerShell, and plugins for code editors such as VS Code, for administrators and developers who prefer these tools.

Infrastructure as code (IaC). Instead of clicking or typing commands one by one, you **describe your whole infrastructure in a text file**, and AWS builds exactly what the file says.

- **AWS CloudFormation**: you write a **template** in JSON or YAML listing the resources (for example "one network, two servers, one database, with these settings"). CloudFormation creates them together as a **stack**, in the right order. You can run the same template in another Region or account and get an **identical** environment; you can update the stack by changing the template; if creation fails, it **rolls back** automatically. CloudFormation itself is free: you pay only for the resources it creates.
- **AWS CDK (Cloud Development Kit)**: the same idea, but you describe the infrastructure in a programming language; the CDK turns it into CloudFormation.

> EXAMPLE
> Sara wants a test copy of her shop, identical to production, to try a new feature. With the console her developer would spend a day clicking and might forget a setting. With a CloudFormation template, he runs it once: in fifteen minutes the test environment exists, identical. When the test is over, he deletes the stack and everything disappears.

One-time operation or repeatable process?

The official objective asks you to choose the right approach:

- **One-time, simple, exploratory** tasks (look at a dashboard, test a service, fix one setting): the **console** is fine.
- **Repeatable** tasks (create the same environment several times, deploy every week, manage many accounts): use **CLI scripts, SDKs or infrastructure as code**. Benefits: speed, **consistency**, fewer human errors, version history, easy review.

ARN (Amazon Resource Name). Each AWS resource has a unique identifier called an ARN, for example `arn:aws:s3:::sara-invoices`. You see ARNs in permissions (IAM policies) and in scripts, to point to an exact resource.

GOOD TO KNOW

Some services are **global** and some are **regional**. In the console, IAM, CloudFront and Route 53 show *Global* at the top right, while EC2 or RDS show the Region you are working in. A classic beginner surprise: you create a server, switch Region by accident, and think it has disappeared. For practice labs, **us-east-1 (North Virginia)** is a good default: it is the oldest Region and offers the most services. AWS CloudShell, a terminal inside the console, is `OUT` in v1.3.

Deployment models, a reminder

The three deployment models were explained in section 1.1: **cloud** (everything on AWS), **hybrid** (on-premises and AWS connected together) and **on-premises** (everything in your own data center). The exam may ask you to identify them from a description: "the company keeps its mainframe in its data center and runs its website on AWS, connected with Direct Connect" is **hybrid**.

ON THE EXAM

"Repeatable, consistent deployments in several Regions" = **CloudFormation** (infrastructure as code). "Automate tasks with scripts" = **CLI**. "Application code calls AWS" = **SDK**. "Web interface, one-time task" = **Management Console**.

3.2 The AWS global infrastructure

Regions, Availability Zones and edge locations

IN PLAIN WORDS

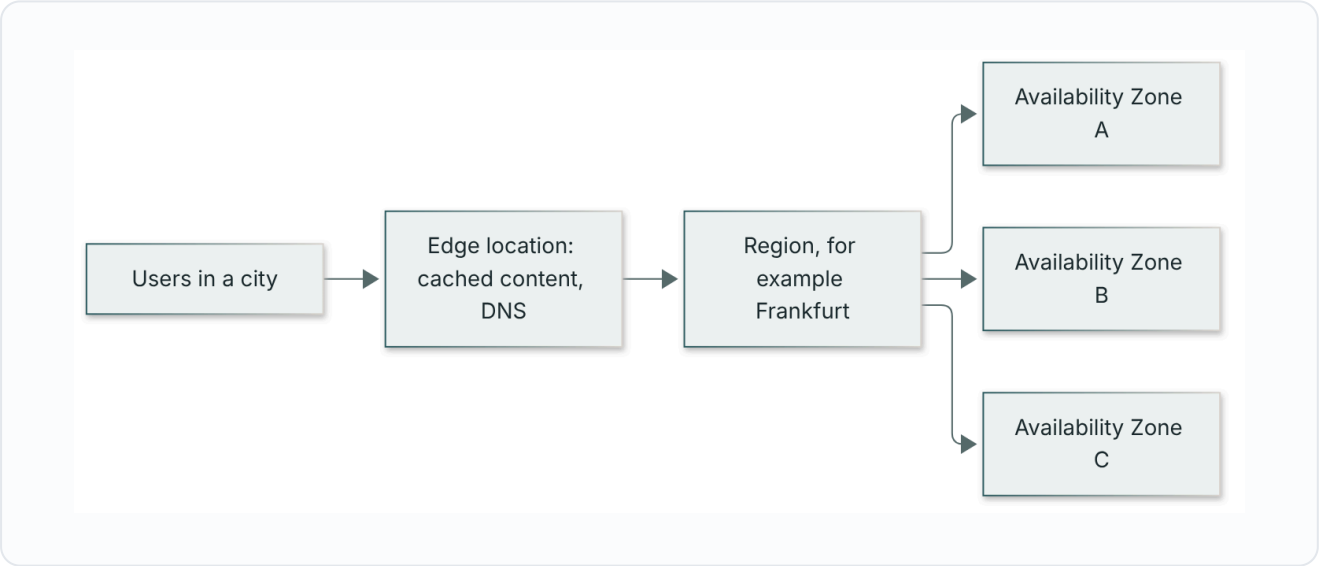
AWS has data centers all over the world. They are organized in three levels, from the largest to the smallest.

Regions. A **Region** is a geographic area of the world where AWS has a group of data centers, for example Ireland (`eu-west-1`), Frankfurt (`eu-central-1`), Paris (`eu-west-3`) or North Virginia (`us-east-1`). There are more than 30 Regions. Key facts:

- Each Region is **completely independent and isolated** from the others: a problem in one Region does not affect the others.
- **Your data stays in the Region you choose.** AWS never moves it to another Region unless you decide to copy it. This matters for laws on where data must be kept.
- You choose the Region for each resource; prices and available services vary between Regions.

Availability Zones (AZs). Each Region contains **several Availability Zones**, at least three in most Regions. An AZ is **one or more data centers**, each with **its own power supply, cooling and network**. The AZs of a Region are physically **separated by a meaningful distance** (kilometres apart) so that a flood, fire or power cut is unlikely to hit two of them, but they are connected by fast, private fibre links with very low latency. The official wording: **Availability Zones do not share single points of failure.** They are named like `eu-west-1a`, `eu-west-1b`, `eu-west-1c`.

Edge locations. Much smaller sites, in **many more cities** than there are Regions (hundreds of them), also called **points of presence**. They do not run your servers. Their job is to bring some services **closer to users**: **Amazon CloudFront** caches copies of your files there, **Amazon Route 53** answers DNS requests there, **AWS Global Accelerator** lets user traffic enter the fast AWS network there.



ANALOGY

A Region is a city. Its Availability Zones are separate neighbourhoods of that city, each with its own power station and water supply, linked by fast motorways. Edge locations are small kiosks in many other towns that keep popular items close to customers.

NOTE

Latency is the time it takes for data to travel between a user and a server, measured in milliseconds. The farther away the server, the higher the latency, and the slower the website feels. Choosing a Region near your users and using edge locations both reduce latency.

Achieving high availability with several Availability Zones

Because AZs do not share points of failure, the basic rule for **high availability** on AWS is: **run your application in at least two Availability Zones.** If one AZ has a problem, the other keeps serving.

EXAMPLE

Sara runs her web servers in two AZs of the Paris Region, with a load balancer in front, and her database has a standby copy in a second AZ (RDS Multi-AZ). A power failure hits AZ A: the load balancer sends all visitors to the servers in AZ B, and the database switches to its standby. Customers notice nothing.

When to use several Regions

One Region with several AZs is enough for high availability in most cases. You use **several Regions** for four reasons listed in the official guide:

1. **Disaster recovery**: survive an event that affects a whole Region.
2. **Business continuity**: keep the business running whatever happens.
3. **Low latency for end users**: serve customers on different continents from a Region near them.
4. **Data sovereignty**: keep each country's data inside that country, as its laws require.

How to choose a Region

1. **Compliance and data residency**: what the law or your customers require, for example health data of German patients kept in Germany. This comes first.
2. **Proximity to users (latency)**: choose the Region closest to most of your users.
3. **Service availability**: not every service or feature exists in every Region; new ones usually arrive first in `us-east-1`.
4. **Pricing**: prices differ from one Region to another.

Other kinds of locations

Name	In plain words
AWS Outposts	AWS installs and manages its own racks of servers **inside your data center**, running AWS services with the same APIs and tools. For data that must stay on site or applications that need very low latency to local systems [v1.3]
AWS GovCloud (US)	Isolated Regions for US government agencies and companies with strict US regulations
AWS China Regions	Beijing and Ningxia, operated by local Chinese partners, isolated from other Regions, with separate accounts
Local Zones LEGACY	Small extensions of a Region in large cities, for single-digit millisecond latency
Wavelength Zones OUT	AWS infrastructure inside 5G mobile networks

Disaster recovery

IN PLAIN WORDS

A **disaster** is any event that makes your application unusable: a whole Region down (very rare), a major mistake, a cyberattack that destroys data. **Disaster recovery (DR)** is your plan to restore service, usually in another Region. Two numbers define what you need:

- **RPO, Recovery Point Objective**: the maximum amount of **data** you accept to lose, expressed in time. "RPO of 1 hour" means losing the last hour of orders is acceptable, so you need copies at least every hour.
- **RTO, Recovery Time Objective**: the maximum time the service can be **down**. "RTO of 15 minutes" means the shop must be back within 15 minutes.

The smaller these numbers, the more you must keep running in the second Region, and the more it costs. AWS describes four strategies, from cheapest and slowest to most expensive and fastest:

Backup and restore. You regularly copy your data (backups, snapshots) to another Region. Nothing else runs there. After a disaster, you rebuild everything from the backups. RPO and RTO: **hours**. Cost: **lowest**.

Pilot light. The **data** is continuously copied to the second Region, and the essential parts of the system are set up but **switched off** or minimal. In a disaster you "light" them up and scale. RPO and RTO: **tens of minutes**.

Warm standby. A **complete but smaller** copy of the whole environment **is always running** in the second Region. In a disaster you simply scale it up to full size and redirect users. RPO and RTO: **minutes**.

Multi-site active/active. The full application **runs at the same time in two or more Regions**, all serving users. If one Region fails, the others take all the traffic. RPO and RTO: **near zero**. Cost: **highest**.

Strategy	Picture it as	Running in the second Region	Recovery
Backup and restore	Photocopies of your documents in a safe	Only backups	Hours
Pilot light	The small flame of a gas boiler, ready to ignite	Data replicated, core systems ready but off	Tens of minutes
Warm standby	A spare car with the engine idling	Small but complete working copy	Minutes
Multi-site active/active	Two shops open at the same time	Full copy serving users	Almost none

Services: **AWS Backup** creates and manages backups centrally (EC2, EBS, RDS, DynamoDB, EFS, FSx...) and can copy them to another Region. **AWS Elastic Disaster Recovery** (formerly CloudEndure Disaster Recovery) continuously replicates your servers into a low-cost staging area in AWS, so you can launch them in minutes after a disaster.

ON THE EXAM

"Region contains several AZs, an AZ is one or more data centers, edge locations are separate" is the core relationship. "High availability" = **multiple AZs**. "Survive a Region-wide disaster", "users on several continents", "data must stay in a country" = **multiple Regions**. "Data stays in its Region unless you move it". "Lowest DR cost, recovery in hours" = backup and restore; "scaled-down copy always running" = warm standby; "near-zero downtime" = multi-site. RPO = data loss, RTO = downtime.

3.3 Compute services

IN PLAIN WORDS

"Compute" is the processing power that runs your applications. AWS offers it in several forms, from the most control (virtual servers) to the least management (serverless). Choosing among them is a frequent exam question.

Amazon EC2: virtual servers

Amazon EC2 (Elastic Compute Cloud) lets you rent **virtual servers**, called **instances**, in minutes. You choose the operating system (Linux, Windows, macOS), the size, the storage and the network settings. You have **full control** of the operating system and everything installed on it, and you are **responsible** for it (it is IaaS: you patch, configure and secure the OS). You can start, stop and terminate instances at any time; Linux instances are billed per second.

To launch an instance, you choose five things:

1. **An AMI (Amazon Machine Image)**: the template containing the operating system and optionally pre-installed software, like an installation disk. AWS, vendors (in Marketplace) and you can provide AMIs.
2. **An instance type**: the amount of CPU, memory, storage speed and network capacity, for example `t3.micro` (tiny, part of the Free Tier) or `m7g.large`.
3. **Storage**: usually an **EBS volume**, a virtual disk (see 3.6).
4. **A security group**: the firewall rules saying which traffic may reach the instance (see 3.5).
5. **A key pair**: the cryptographic key used to connect securely to the instance (SSH for Linux).

Instance families. Instance types are grouped in families, each optimized for a kind of workload. The first letter of the name gives the family.

Family	Optimized for	Typical workloads
General purpose (T, M)	Balance of CPU, memory and network	Web servers, small and medium databases, development environments
Compute optimized (C)	High-performance processors	Batch processing, video encoding, gaming servers, scientific modeling, high performance computing (HPC), machine learning inference
Memory optimized (R, X)	Large amounts of memory	In-memory databases and caches, real-time big data analytics
Storage optimized (I, D)	Very fast, high-throughput local storage, high IOPS	Large transactional databases, data warehouses, distributed file systems
Accelerated computing (P, G, Inf, Trn)	GPUs or specialized machine learning chips	Training ML models, graphics rendering, video processing

NOTE

IOPS (input/output operations per second) measures how many small reads and writes a disk can do each second. Databases do huge numbers of small reads and writes, so they need high IOPS: that is the job of **storage optimized** instances and fast SSD volumes.

GOOD TO KNOW

AWS designs its own server technology, the **Nitro System** (dedicated hardware cards and a lightweight hypervisor), used by all recent instance types for performance and security. You can even rent **bare metal** instances, with no hypervisor at all, when software must run directly on the hardware. For the exam, just recognize the names.

> GOOD TO KNOW
> **AWS Graviton** processors are ARM-based chips designed by AWS (instance names with a **g**, such as m7g or c7g). They give the **best price-performance** for many workloads and use **less energy**: a frequent answer for "reduce cost" and "sustainability".

Placement groups decide **how instances are placed on the hardware** (free of charge): **cluster** (close together in one AZ, lowest latency between instances, for high-performance computing), **spread** (each instance on separate hardware, so they never fail together) and **partition** (groups of instances on separate racks, for big distributed systems such as Hadoop or Kafka).

Elastic Load Balancing: spreading the traffic

The problem. One server can handle only so many visitors, and if it fails, the site is down. So we run several servers. But then, how do visitors reach them, and how do we avoid sending people to a broken one?

The solution. A **load balancer** is placed in front of the servers. Visitors connect to the load balancer, and it **distributes the requests** among the servers. It continuously runs **health checks** (small test requests) and **stops sending traffic to unhealthy servers**. It can spread traffic across several **Availability Zones**, which gives high availability. On AWS, the service is **Elastic Load Balancing (ELB)**, fully managed and scaling automatically.

Types of load balancer:

- **Application Load Balancer (ALB)**: understands web traffic (HTTP and HTTPS, "layer 7"). It can route requests by their content, for example `/images` to one group of servers and `/api` to another. The usual choice for websites and web applications.
- **Network Load Balancer (NLB)**: works at the network level (TCP/UDP, "layer 4"), handles millions of requests per second with very low latency, and supports static IP addresses. For extreme performance or non-web protocols.
- **Gateway Load Balancer (GWLB)**: sends traffic through a fleet of third-party virtual appliances such as firewalls.
- Classic Load Balancer: the old generation.

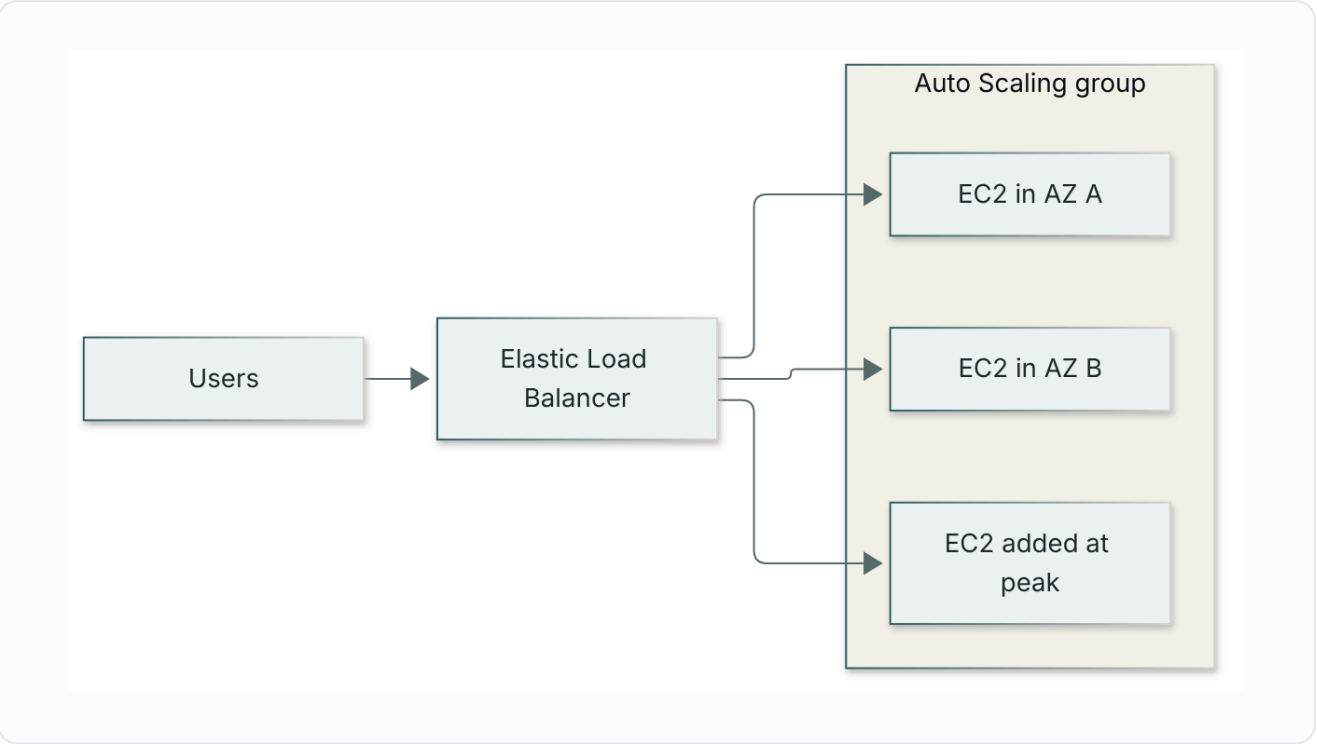
Amazon EC2 Auto Scaling: elasticity

The problem. Sara's traffic changes: quiet at night, busy at lunchtime, enormous on Mother's Day. Fixed capacity is either too small (crash) or too big (waste).

The solution. Amazon EC2 Auto Scaling automatically **adds instances when demand rises and removes them when it falls**. You create an **Auto Scaling group** with three numbers: a **minimum** (never fewer, for availability), a **desired** capacity, and a **maximum** (never more, to control cost). Scaling can follow:

- a **metric** (for example, keep average CPU around 50%: dynamic scaling);
- a **schedule** (for example, 10 servers every Saturday morning: scheduled scaling);
- a **forecast** based on past patterns (predictive scaling).

Auto Scaling also **replaces unhealthy instances** automatically. It is free; you pay for the instances. **Auto Scaling is what gives elasticity** to EC2, a fact the exam asks often.



> ANALOGY: A RESTAURANT
>
> The **load balancer** is the host at the entrance who sends each customer to a free table, in several dining rooms. **Auto Scaling** is the manager who calls in extra waiters when the restaurant fills up and sends them home when it is quiet.

Containers: ECS, EKS, ECR and Fargate

The problem. A developer's application works on their laptop but fails on the server because a library version is different. And virtual machines are heavy: each carries a full operating system.

The solution. A **container** packages an application **with everything it needs** (code, libraries, settings) into one standard unit that runs the same way everywhere. Containers on the same host **share its operating system**, so they are much lighter and faster to start than virtual machines; one server can run many containers. **Docker** is the best-known container technology. Large applications are often split into many small services, each in its own container (**microservices**).

ANALOGY

Shipping containers: whatever is inside, every ship, crane and truck in the world handles the same standard box. Software containers do the same for applications.

Running many containers needs an **orchestrator**: a system that decides where each container runs, restarts failed ones, and scales them. On AWS:

- **Amazon ECS (Elastic Container Service)**: AWS's own container orchestrator, simple and deeply integrated with AWS.
- **Amazon EKS (Elastic Kubernetes Service)**: runs **Kubernetes**, the very popular open-source orchestrator, managed by AWS. Choose it when the team already uses Kubernetes or wants portability.
- **Amazon ECR (Elastic Container Registry)**: a private **library where you store your container images**, ready to be deployed.
- **AWS Fargate**: a **serverless engine for containers**. With ECS or EKS you normally need EC2 instances to run the containers. With Fargate you do not manage any server: you say "run this container with this much CPU and memory", and AWS does the rest. You pay for the resources the container uses.

Serverless: AWS Lambda

IN PLAIN WORDS

Serverless does not mean there are no servers. It means **you never see, manage, patch or size them**. AWS runs them, scales automatically from zero to thousands of requests, and you **pay only when your code runs**.

AWS Lambda runs small pieces of code, called **functions**, **in response to events**: a file uploaded to S3, a request coming through API Gateway, a message in a queue, a change in a database, or a schedule ("every night at 2 a.m."). You upload your code (Python, Node.js, Java...), set the memory, and Lambda handles everything else. You pay per **request** and per **millisecond** of execution. A single run can last **up to 15 minutes**, so Lambda suits short tasks, not long-running programs.

EXAMPLE

A customer uploads a photo of the cake she wants copied. S3 triggers a Lambda function, which creates a small preview image and stores it. Between uploads, nothing runs and nothing is paid.

ANALOGY

An EC2 instance is like owning a car: it costs money even when parked. Lambda is like a taxi: you pay only for the journey.

Other compute services

AWS Elastic Beanstalk. A **platform as a service**: developers upload their application code (Java, .NET, PHP, Node.js, Python, Ruby, Go, Docker), and Beanstalk **automatically creates and manages** the EC2 instances, load balancer, Auto Scaling and monitoring. You keep access to the underlying resources if needed. Beanstalk is free; you pay for the resources.

Amazon Lightsail. Simple virtual private servers at a **fixed, low monthly price** that includes storage and a data transfer allowance, with an easy interface. Ideal for small websites, blogs (WordPress), simple apps, and beginners who want a predictable bill.

AWS Batch. Runs **batch computing jobs** (thousands of processing tasks such as simulations, rendering, analysis) at any scale. It decides how many and which instances to use, can use cheap Spot Instances, and queues the jobs.

AWS Outposts. AWS compute and other services **in your own data center** (see 3.2), including **S3 on Outposts** for data that must stay on site.

AWS App Runner. Deploys a **web application or API directly from source code or a container image**: App Runner builds it, runs it, load-balances it and scales it, with no infrastructure to configure.

If the question says	Choose
Full control of the operating system, custom software	**EC2**
Spread traffic across instances and AZs, health checks	**Elastic Load Balancing**
Automatically add and remove instances (elasticity)	**EC2 Auto Scaling**
Run containers with AWS's orchestrator	**ECS**
Kubernetes	**EKS**
Containers without managing servers	**Fargate**
Store container images	**ECR**
Run code on events, no servers, pay per request	**Lambda**
Developers upload code, AWS handles deployment	**Elastic Beanstalk**
Simple server, fixed monthly price	**Lightsail**
Large numbers of batch jobs	**Batch**

ON THE EXAM

"Elasticity" = **Auto Scaling**. "Distribute traffic, route around unhealthy instances" = **load balancer**. "Compute optimized" for CPU-heavy work, "memory optimized" for in-memory databases, "storage optimized" for high IOPS. "Serverless" options = **Lambda** (functions) and **Fargate** (containers).

3.4 Database services

What a database is

IN PLAIN WORDS

A **database** is software that stores data in an organized way so that applications can save, find and update it quickly and safely, even when thousands of users do it at the same time. Sara's shop needs one for customers, products, orders and payments.

There are two big families.

Relational databases (SQL). Data is stored in **tables** with rows and columns, like linked spreadsheets: a *Customers* table, an *Orders* table, a *Products* table, connected by **relations** (each order points to a customer). You query them with the **SQL** language. They enforce a strict structure (a **schema**) and guarantee consistency: an order is either fully saved or not at all (**transactions**). Perfect for orders, payments, accounting. Examples of engines: MySQL, PostgreSQL, MariaDB, Oracle, Microsoft SQL Server.

Non-relational databases (NoSQL). Data is stored in more flexible formats, without a fixed table structure:

- **Key-value**: each item is found by a unique key, like a giant dictionary ("cart-12345" → contents of the cart). Extremely fast and scalable.
- **Document**: each item is a document (usually JSON) that can have its own fields, convenient for product catalogs or user profiles.
- **Graph**: stores items and the **relationships** between them (who knows whom, who bought what), for social networks, recommendations, fraud detection.
- **In-memory**: data kept in RAM for microsecond access, mostly used as a **cache**.

NoSQL databases scale to enormous sizes and traffic and handle changing data structures easily.

Managed or self-managed?

You can install a database yourself on an **EC2 instance**: you get **full control** (any engine, any version, any setting), but you must install it, patch the OS and the database, configure backups, replication and failover, and scale it. Or you use an **AWS managed database**: AWS handles provisioning, patching, backups, high availability and scaling; you focus on your data and queries.

Choose **EC2-hosted** when you need something a managed service does not offer (a special engine or version, OS-level access). Otherwise choose **managed**: it is the answer the exam usually expects, because it reduces operational work.

The AWS database services

Amazon RDS (Relational Database Service). Managed relational databases with your choice of engine: **MySQL, PostgreSQL, MariaDB, Oracle, Microsoft SQL Server, IBM Db2**. AWS handles installation, OS and engine patching, **automated backups** with point-in-time recovery, monitoring and failover.

Amazon Aurora. AWS's own relational engine, **compatible with MySQL and PostgreSQL** (applications work without changes), designed for the cloud: much faster than standard MySQL, highly available, storing **six copies of your data across three AZs**, and growing storage automatically. Choose it for demanding relational workloads. **Aurora Serverless** adjusts capacity automatically.

Amazon DynamoDB. A **serverless NoSQL key-value and document database**. No servers to manage, automatic scaling, and **single-digit millisecond** response times **at any scale**, from ten users to millions. Used for shopping carts, user sessions, gaming leaderboards, IoT data. **DynamoDB global tables** replicate a table automatically across several Regions (users read and write in the nearest one), and **DynamoDB Accelerator (DAX)** is an in-memory cache that brings reads down to **microseconds**.

Amazon ElastiCache. An **in-memory** data store (Redis-compatible engines or Memcached). Reading from memory takes **microseconds**. Its main use is as a **cache** in front of a database: frequently requested results (the list of best-selling cakes) are kept in memory, so the database is not asked the same question a thousand times per second. Result: faster pages and less load on the database.

Amazon DocumentDB NEW. A managed **document** database, **compatible with MongoDB**: ideal when a team uses MongoDB and wants AWS to manage it.

Amazon Neptune. A managed **graph** database for highly connected data: social networks, recommendation engines ("customers who bought this also bought..."), fraud detection, knowledge graphs.

Amazon Redshift. A **data warehouse**: a database designed to analyze **huge amounts of historical data** (up to petabytes) with SQL, for reports and business intelligence.

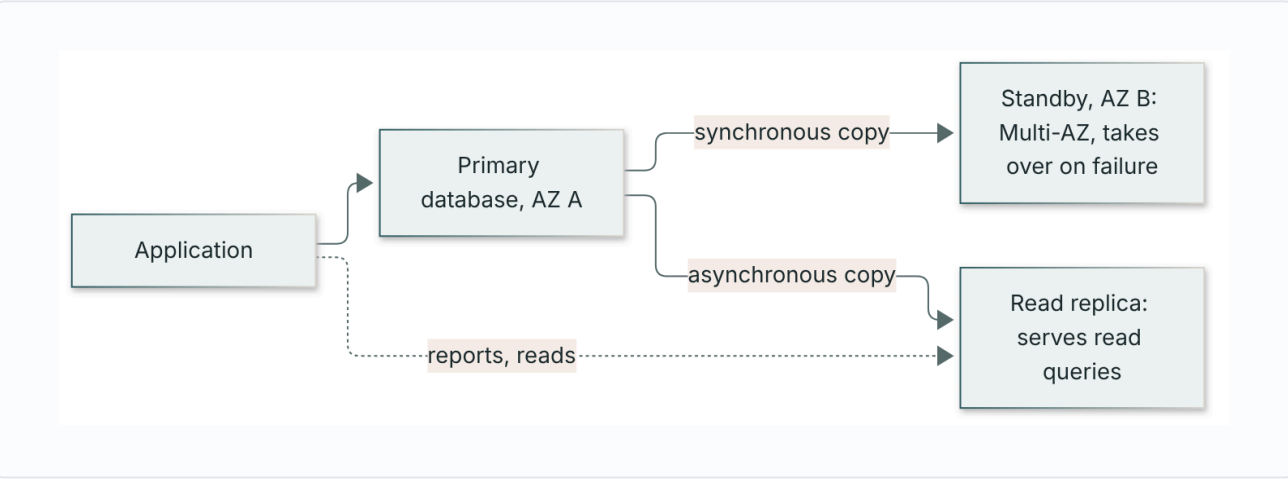
Out of scope: Amazon Keyspaces OUT (Apache Cassandra compatible), Amazon MemoryDB OUT (durable in-memory database compatible with Redis), **Amazon Timestream** OUT (a **time series** database for data points with a timestamp: IoT sensor readings, application metrics). Amazon QLDB LEGACY (a ledger database with a verifiable history of changes) reached its end of support in 2025.

> **NOTE**
> Two types of database work. **OLTP** (online transaction processing): many small, fast operations by users, such as placing an order; served by RDS, Aurora, DynamoDB. **OLAP** (online analytical processing): few but huge queries over years of data, such as "sales by region over five years"; served by a data warehouse, **Redshift**.

Multi-AZ and read replicas

Two RDS features are often confused:

- **Multi-AZ deployment** = **high availability**. RDS keeps a **standby copy** of the database in another AZ, updated **synchronously**. If the main database fails, RDS **automatically switches** to the standby. The standby does not serve traffic; it waits.
- **Read replicas** = **performance**. Extra copies, updated asynchronously, that **answer read queries** (reports, product pages) to take load off the main database. They improve read scalability.



Moving databases to AWS

- **AWS Database Migration Service (DMS)** copies a database to AWS while the source **keeps running**, then continuously replicates new changes until you switch over: **minimal downtime**. It works between the same engine (MySQL to MySQL) or different engines (Oracle to Aurora).
- **AWS Schema Conversion Tool (SCT)** converts the **schema** (tables, views, stored procedures) when you change engine. SCT converts the structure; DMS moves the data.

If the question says	Choose
Managed relational database, choice of engine	**RDS**
MySQL/PostgreSQL compatible, highest performance and availability	**Aurora**
NoSQL, key-value, millisecond latency at any scale, serverless	**DynamoDB**

If the question says	Choose
Cache, microsecond reads, reduce database load	**ElastiCache**
MongoDB compatible	**DocumentDB**
Relationships, social graph, recommendations	**Neptune**
Data warehouse, analytics on huge historical data	**Redshift**
Full control of the database software and OS	Database on **EC2**
High availability of an RDS database	**Multi-AZ**
Improve read performance	**Read replicas**
Migrate a database with minimal downtime	**DMS** (+ **SCT** if the engine changes)

3.5 Network services

Amazon VPC: your private network in AWS

IN PLAIN WORDS

When Sara launches servers and a database, they must be connected to a network, and she must decide what can be reached from the internet and what must stay hidden. **Amazon VPC (Virtual Private Cloud)** gives her **her own logically isolated network** inside AWS, in one Region. Nobody else can see into it. She chooses its range of private IP addresses and divides it into smaller networks. VPCs themselves are free.

> ANALOGY
> The VPC is your private floor in a big office tower shared with other companies. **Subnets** are the rooms of your floor. The **public subnet** is the reception area, where visitors may come. The **private subnet** is the back office, closed to visitors. The **internet gateway** is the main door of your floor. The **NAT gateway** is the mail room: back-office staff can send letters out, but strangers cannot walk in through it.

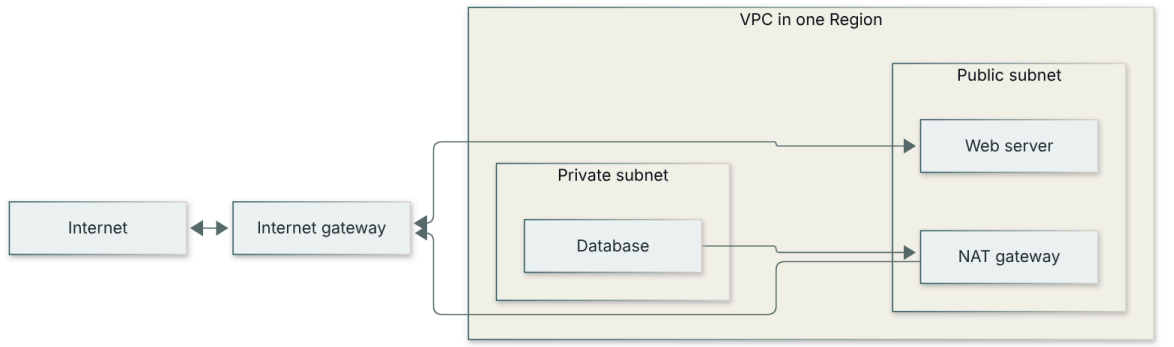
The components:

- **Subnets.** A subnet is a section of the VPC's address range, placed in **one Availability Zone**. For high availability you create subnets in several AZs.
  - A **public subnet** has a route to the internet gateway: resources there (such as a web server or a load balancer) can be reached from the internet.
  - A **private subnet** has no direct route from the internet: resources there (such as the database) are hidden.

Each EC2 instance connects to its subnet through an **elastic network interface (ENI)**: a virtual network card with a private IP address (and optionally a public one). Security groups are

attached to it.

- **Internet gateway.** Connects the VPC to the internet, in both directions, for resources in public subnets.
- **NAT gateway.** Lets resources in a **private** subnet **start connections to the internet** (to download updates, for example) while remaining **unreachable from the internet**.
- **Route tables.** Sets of rules that decide where network traffic from each subnet is sent (to the internet gateway, to the NAT gateway, to another network).
- **Security groups and network ACLs.** The two firewalls (explained below).



EXAMPLE

Sara's architecture: the load balancer and web servers sit in **public subnets** in two AZs; the database sits in **private subnets**, so nobody on the internet can reach it directly; only the web servers can talk to it. The database server downloads its updates through the NAT gateway.

Security in a VPC: security groups and network ACLs

Two firewalls protect resources in a VPC, at different levels.

Security groups act at the level of **each instance** (more precisely, its network interface). You write **allow** rules only, for example "allow HTTPS from anywhere", "allow SSH only from my office's IP", "allow the database port only from the web servers' security group". Everything not allowed is blocked. Security groups are **stateful**: if a request is allowed in, the reply is automatically allowed out, without a separate rule.

Network ACLs (access control lists) act at the level of a **whole subnet**, as a second barrier. They have **allow and deny** rules, evaluated in order by number. They are **stateless**: incoming and outgoing traffic are checked separately, so you must allow the reply traffic explicitly.

ANALOGY

The **network ACL** is the guard at the gate of the residence: he checks everyone entering **and** leaving the whole residence, and he has a list of people allowed and a list of people **banned**. The **security group** is the lock of your own apartment door: it only lets in people on your guest list, and once a guest is in, they can leave freely.

	Security group	Network ACL
Protects	An **instance**	A **subnet**
Rules	**Allow** only	**Allow and deny**
State	**Stateful**: replies are allowed automatically	**Stateless**: in and out checked separately
Typical use	"Allow web traffic, SSH from my IP"	"**Block this malicious IP address**"

GOOD TO KNOW

The classic example to separate the two: you want to **block one specific IP address**. A security group cannot do it, because it only has allow rules (you would have to allow every other address one by one). A network ACL can, with a single deny rule.

The official objective also mentions **Amazon Inspector** for VPC security: it reports instances that are reachable from the internet in unintended ways (open ports) as well as software vulnerabilities (see 2.4).

Connecting networks together

- **VPC peering**: a private connection between **two VPCs** (even in different accounts or Regions), so their resources communicate as if on the same network.
- **AWS Transit Gateway** NEW: a **central hub** that connects **many VPCs and on-premises networks** together. With ten VPCs, peering every pair would need dozens of connections; with Transit Gateway each network connects once to the hub.
- **AWS PrivateLink** (VPC endpoints) NEW: lets resources in your VPC reach AWS services, or services offered by other companies, **privately**, without going through the internet. Traffic stays on the AWS network, which improves security.

Connecting your office or data center to AWS

Public internet. Simply use AWS over the normal internet, with encryption (HTTPS). Simple, but performance varies with internet conditions.

AWS Site-to-Site VPN. Creates an **encrypted tunnel** (IPsec) over the internet between your company network and your VPC. Your office network and your AWS network behave like one. **Quick to set up** (minutes to hours), **low cost**, **encrypted**, but performance depends on the internet.

AWS Client VPN. The same idea for **individual users**: employees connect from their laptops, from anywhere, to the company's AWS and on-premises networks.

AWS Direct Connect. A **dedicated, private physical connection** between your data center and AWS, through a **Direct Connect location** (a partner data center where AWS has equipment). Traffic **does not travel over the public internet**, so performance is **consistent**, bandwidth is high, and data transfer out costs less. It takes **weeks** to install and costs more. Direct Connect is **not encrypted by default**: for encryption, run a VPN over it.

> ANALOGY
> A VPN is an armoured truck on the public motorway: protected, but stuck in the same traffic as everyone. Direct Connect is your own private road.

Reaching users around the world

Amazon Route 53: DNS. Computers find each other with IP addresses (like `52.95.110.1`), but people type names (`www.saras-bakery.com`). **DNS (Domain Name System)** is the internet's phone book that translates names into addresses. **Route 53** is AWS's highly available DNS service. It can also:

- **register domain names** (buy `saras-bakery.com`);
- **check the health** of your servers;
- **route users** intelligently: to the Region with the **lowest latency** for them, by their **geographic location**, by **weights** (90% to the old version, 10% to the new), or to a backup site if the main one fails (**failover**).

Amazon CloudFront: content delivery network (CDN). If Sara's servers are in Paris, a customer in Tokyo waits longer for every photo. CloudFront **caches copies** of her images, videos, pages and files in **edge locations** around the world. The Tokyo customer receives them from a nearby edge location: much faster. It also reduces the load on Sara's servers and integrates with **AWS Shield** and **AWS WAF** for security. Between the edge locations and your servers, larger **regional edge caches** keep less popular content for longer, so even fewer requests reach the origin.

AWS Global Accelerator. Gives your application **two fixed (static) IP addresses** reachable worldwide. Users' traffic enters the AWS network at the nearest edge location and travels on AWS's fast private network to the best Region, instead of crossing the public internet. It improves performance and availability for **any TCP or UDP traffic**, not only websites (games, voice, IoT).

Amazon API Gateway. Lets developers create, publish, secure and monitor **APIs**, the front doors through which apps (mobile apps, websites) call the back end. Often placed in front of Lambda functions.

If the question says	Choose
Isolated private network in AWS	**VPC**
Private subnet must download updates from the internet	**NAT gateway**
Instance-level firewall, stateful, allow rules	**Security group**
Subnet-level firewall, stateless, can deny an IP	**Network ACL**
Connect many VPCs and on-premises networks through a hub	**Transit Gateway**
Reach a service privately without the internet	**PrivateLink**
Encrypted connection over the internet, quick to set up	**Site-to-Site VPN**
Dedicated private connection, consistent performance	**Direct Connect**
DNS, domain registration, latency or failover routing	**Route 53**
Cache content near users, CDN	**CloudFront**
Static IPs, faster global routing for any traffic	**Global Accelerator**
Create and secure APIs	**API Gateway**

3.6 Storage services

Three ways to store data

IN PLAIN WORDS

Data can be stored in three different ways, and AWS has a service for each.

Type	How it works	Picture it as	AWS service
Block storage	Data is split into fixed-size blocks on a disk that is attached to **one server**; the operating system formats it and uses it like a hard disk	The hard disk inside a computer	**Amazon EBS**, instance store
File storage	Files are organized in folders and shared over the network; **many servers** can open the same files at the same time	A shared network drive in an office	**Amazon EFS**, **Amazon FSx**
Object storage	Each file is stored as an **object** with its data, metadata and a unique key, in a flat space, and reached over HTTP through an API	A valet storage service: hand over a labelled box, get it back with the label, never mind where it is kept	**Amazon S3**

Amazon S3: object storage

Amazon S3 (Simple Storage Service) stores any amount of data as **objects** inside containers called **buckets**.

- An **object** is a file plus its **metadata** (information about it) and a **key** (its name, for example `photos/cakes/chocolate.jpg`). An object can be from **0 bytes up to 5 TB**.
- A **bucket** holds objects. Bucket names must be **globally unique** across all AWS accounts (like domain names), but each bucket lives in **one Region** that you choose.
- Storage is **virtually unlimited**: you never provision space, you pay for what you store.
- S3 is designed for **eleven nines of durability** (99.999999999%): objects are automatically stored on multiple devices across at least three AZs (except One Zone classes).
- Everything is **private by default**. You grant access with bucket policies and IAM.
- **Versioning** keeps every version of an object, so you can recover from accidental deletion or overwriting.
- **S3 Object Lock** stores objects in **WORM** mode (write once, read many): during a retention period nobody can delete or change them, which meets legal retention rules and protects against ransomware.
- **Replication** copies new objects automatically to another bucket: **Cross-Region Replication (CRR)** for disaster recovery, compliance or lower latency in another Region, **Same-Region Replication (SRR)** inside the same Region.
- **S3 Transfer Acceleration** speeds up **long-distance uploads**: data enters the nearest edge location, then travels over the AWS private network to the bucket.
- Typical uses: website images and videos, **backups and archives**, log files, data lakes for analytics, **static website hosting**, software distribution.

> GOOD TO KNOW
> An S3 object can even be **0 bytes**, and the maximum is **50 TB** per object (raised from 5 TB at the end of 2025). S3 now also offers an account-level naming option that avoids bucket name clashes, but the exam still expects: **bucket names are globally unique**.

S3 storage classes

The idea. Not all data is used equally. Photos on Sara's homepage are viewed thousands of times a day; invoices from five years ago are almost never opened but must be kept by law. S3 offers **storage classes**: the less often you need the data, the **cheaper it is to store**, but the more it costs or the longer it takes to **retrieve** it.

> ANALOGY: YOUR CLOTHES
> Everyday clothes are in the wardrobe (Standard). Winter clothes are in the cupboard upstairs (Infrequent Access). Old souvenirs are in a box at your parents' house (Glacier). Documents you keep only because the law says so are in a storage unit outside town (Glacier Deep Archive).

S3 Standard. For **frequently accessed** data. Fast, highly available, stored in at least three AZs. The default class.

S3 Intelligent-Tiering. For data whose access pattern is **unknown or changes**. S3 **automatically moves** each object between access tiers based on its use, with no retrieval fee and no performance impact; you pay a small monitoring fee per object. When you do not know, choose this.

S3 Standard-Infrequent Access (Standard-IA). For data accessed **less than about once a month** but needed **immediately** when requested (backups, older documents). Cheaper storage than Standard, but a **fee per GB retrieved**, and a minimum storage duration of 30 days.

S3 One Zone-Infrequent Access (One Zone-IA). Like Standard-IA but stored in **a single AZ**, so about 20% cheaper. If that AZ is destroyed, the data is lost. Use it only for data you can **re-create** (for example, copies or thumbnails).

S3 Glacier Instant Retrieval. For **archives** accessed rarely (for example a few times a year) but that must come back **in milliseconds** (old medical images, news archives). Minimum 90 days.

S3 Express One Zone. A **high-performance** class stored in **one Availability Zone**, with single-digit millisecond access, for the most frequently used data (for example machine learning training data). It is about speed, not about saving money.

S3 Glacier Flexible Retrieval (formerly S3 Glacier). For archives where waiting **minutes to hours** for retrieval is acceptable. Cheaper. Minimum 90 days.

S3 Glacier Deep Archive. The **cheapest** storage in AWS, for data kept for **years** for compliance and almost never read (financial records kept 7 to 10 years). Retrieval within **12 hours** (up to 48 hours for bulk). Minimum 180 days.

Class	Access pattern	Retrieval	Note
Standard	Frequent	Immediate	Default, 3+ AZs
Intelligent-Tiering	Unknown or changing	Immediate	Automatic tiering, monitoring fee
Standard-IA	Monthly or less	Immediate, with fee	30-day minimum
One Zone-IA	Monthly or less, re-creatable	Immediate, with fee	**One AZ only**
Glacier Instant Retrieval	A few times a year	Milliseconds	90-day minimum
Glacier Flexible Retrieval	Rare	Minutes to hours	90-day minimum
Glacier Deep Archive	Almost never	Within 12 to 48 hours	**Cheapest**, 180-day minimum

S3 Lifecycle policies. Rules that **automatically move objects to cheaper classes or delete them** as they age. Example: "move order receipts to Standard-IA after 30 days, to Glacier

Flexible Retrieval after 1 year, and delete them after 10 years". Use case: reduce cost without manual work, and apply retention rules automatically.

Block storage: Amazon EBS and instance store

Amazon EBS (Elastic Block Store) provides **virtual hard disks (volumes)** that you attach to EC2 instances.

- **Persistent**: data stays when the instance is stopped, and the volume can be detached and attached to another instance.
- A volume lives in **one Availability Zone** and is normally attached to **one instance** at a time.
- **Snapshots** are point-in-time backups of a volume, stored in S3; you can restore them or copy them to another Region.
- Types: SSD volumes (**general purpose gp3**, or **Provisioned IOPS** io2 for databases that need high, consistent performance) and cheaper HDD volumes for large sequential data.

Instance store is storage on disks **physically attached to the host machine** running your instance. It is very fast, but **temporary (ephemeral)**: data is **lost** when the instance is stopped, terminated, or the hardware fails. Use it only for temporary data: caches, buffers, scratch files.

File storage: Amazon EFS and Amazon FSx

Amazon EFS (Elastic File System) is a fully managed **shared file system for Linux** (NFS protocol). **Many EC2 instances**, in several AZs, can mount it and use the **same files at the same time**. It **grows and shrinks automatically** as you add and remove files; you pay for what you store. Use it when several servers need shared files, such as website content or shared documents.

Amazon FSx provides managed file systems built on popular commercial or high-performance technologies:

- **FSx for Windows File Server**: Windows file shares (SMB protocol) integrated with Microsoft **Active Directory**, for Windows applications.
- **FSx for NetApp ONTAP** and **FSx for OpenZFS**: for companies using these technologies on-premises.
- FSx for Lustre `OUT`: high-performance computing.

Hybrid storage and backups

AWS Storage Gateway connects **on-premises applications** to **cloud storage**. It installs as a virtual machine (or appliance) in your data center, keeps a **local cache** of frequently used data for fast access, and stores the data in AWS. That is why the guide calls it a **cached file system**. Three types:

- **File Gateway**: your servers see a normal network file share; files are stored as objects in S3.

- **Volume Gateway**: your servers see disks; data is backed up to AWS (cached mode keeps the most used data locally, the rest in AWS).
- **Tape Gateway**: replaces physical backup tapes with **virtual tapes** stored in S3 and Glacier, without changing existing backup software.

AWS Backup is a central service to **define backup plans** (how often, how long to keep, where to copy) and apply them across many services: EC2, EBS, RDS, Aurora, DynamoDB, EFS, FSx, Storage Gateway and more, even across accounts and Regions. Use case: one place to make sure everything is backed up and to prove it to auditors.

AWS Elastic Disaster Recovery replicates whole servers for fast recovery (see 3.2).

If the question says	Choose
Store unlimited files, images, backups, static website	**S3**
Unknown or changing access pattern	**S3 Intelligent-Tiering**
Infrequent but immediate access	**S3 Standard-IA**
Infrequent, re-creatable, cheapest IA	**S3 One Zone-IA**
Cheapest long-term archive, hours to retrieve	**S3 Glacier Deep Archive**
Move objects to cheaper classes automatically as they age	**Lifecycle policy**
Persistent disk for one EC2 instance	**EBS**
Temporary, very fast disk, lost on stop	**Instance store**
Shared file system for many Linux instances	**EFS**
Windows file shares with Active Directory	**FSx for Windows File Server**
On-premises applications using cloud storage with a local cache	**Storage Gateway**
Central backup plans across services	**AWS Backup**

3.7 AI/ML and analytics services

First, the vocabulary

Artificial intelligence (AI) is the broad idea of computers performing tasks that normally need human intelligence: understanding speech, recognizing images, translating, deciding. **Machine learning (ML)** is the main way to build AI today: instead of writing rules by hand, you show the system many examples (data) and it **learns patterns**; the result is a **model** that makes predictions on new data. **Generative AI** is a kind of AI that **creates new content**, such as text, images or code, from instructions.

AWS offers AI in two ways:

- **Ready-made AI services**: you send data through an API and get results. **No ML knowledge needed.**
- **A platform to build your own models**: for data scientists who train models on their own data (Amazon SageMaker AI).

The AI services, by task

Amazon SageMaker AI NEW (formerly Amazon SageMaker). The platform to **build, train and deploy your own machine learning models** at scale: prepare data, train, tune, deploy, monitor. *Sara's case: predict how many cakes to bake each day from past sales and the weather.*

Amazon Q NEW. A **generative AI assistant**: it answers questions, summarizes documents and helps with tasks using a company's own data (Amazon Q Business), and helps developers write, explain and fix code and work with AWS (Amazon Q Developer).

Amazon Lex. Builds **chatbots and voice bots** that understand natural language, with the same technology as Alexa. *Sara adds a chatbot that answers "Where is my order?".*

Amazon Polly. Text to speech: turns written text into lifelike spoken audio, in many languages and voices.

Amazon Transcribe. Speech to text: turns audio (calls, videos, meetings) into written text, for subtitles or call analysis.

Amazon Translate. Translates text between languages automatically. *Sara's website in French, English and Arabic.*

Amazon Comprehend. Understands text with natural language processing: detects the **sentiment** (positive or negative), key phrases, people, places and the language. *Sara analyzes thousands of customer reviews.*

Amazon Rekognition. Analyzes images and videos: detects objects, scenes, faces, text, and inappropriate content.

Amazon Textract. Extracts text and data from scanned documents, including forms and tables (invoices, ID cards), going beyond simple character recognition.

Older or out-of-scope names: Amazon Kendra LEGACY (intelligent enterprise search); Amazon Personalize, Amazon Fraud Detector, Amazon CodeGuru OUT.

> **NOTE**
> A memory aid: Polly **speaks**, Transcribe **writes down**, Translate **translates**, Comprehend **understands**, Rekognition **sees**, Textract **reads documents**, Lex **chats**, SageMaker AI **builds your own model**, Q **assists**.

Analytics: turning data into answers

IN PLAIN WORDS

Companies collect huge amounts of data: sales, website clicks, logs, sensor readings. **Analytics** means collecting, preparing, storing, querying and visualizing this data to make decisions. Picture a pipeline: data **arrives** (streaming), is **prepared** (ETL), **stored** (data lake in S3 or a data warehouse), **queried**, and **visualized** in dashboards.

Amazon Kinesis. Collects and processes **real-time streaming data**: website clickstreams, IoT sensor data, application logs, as it arrives, second by second. Use it when data must be analyzed **live**. The family has three members: **Kinesis Data Streams** (a real-time stream that your own applications read and process, with replay), **Amazon Data Firehose** (formerly Kinesis Data Firehose: the simplest way to **load** streaming data into S3, Redshift or OpenSearch, fully managed) and **Kinesis Video Streams** (ingests **video** from cameras for playback and analysis).

AWS Glue. A serverless **ETL** service (**extract, transform, load**): it finds data in various sources, cleans and transforms it, and loads it where it will be analyzed. It also keeps a **data catalog** describing where data is and how it is structured. **AWS Glue Studio** lets you build these ETL jobs visually, and **AWS Glue DataBrew** cleans and normalizes data with clicks, without code.

AWS Lake Formation. Builds and **secures a data lake** on S3 in days instead of months, with **central, fine-grained permissions** (which user can see which table, column or row) for all the analytics services.

Amazon Athena. Run **SQL queries directly on files stored in S3** (CSV, JSON, Parquet), without loading them into a database and without servers. You pay per query (for the data scanned). *Sara queries a year of website logs in S3 in seconds.*

Amazon EMR. Managed clusters running big data frameworks such as **Apache Spark and Hadoop**, to process petabytes of data.

Amazon Redshift. The **data warehouse**: stores structured historical data for fast SQL analytics and reporting (see 3.4). **Redshift Serverless** runs it without managing clusters: capacity scales automatically and you pay for what you use. With **Redshift ML** (and **Aurora ML** for Aurora), analysts can create and use machine learning models with simple SQL.

Amazon OpenSearch Service. Search and log analytics: full-text search for websites and apps, and real-time analysis of logs and metrics.

AWS Data Exchange OUT: find and subscribe to **third-party datasets** (financial, weather, healthcare data) delivered into your AWS account. **Amazon AppFlow** OUT: moves data between **SaaS applications** (Salesforce, Slack, Zendesk) and AWS services without code.

Amazon Quick NEW (formerly Amazon QuickSight). **Business intelligence**: interactive **dashboards and charts** for managers, built on your data, with AI features to ask questions in natural language.

If the question says	Choose
Build, train and deploy your own ML model	**SageMaker AI**
Generative AI assistant for business or developers	**Amazon Q**
Chatbot, conversational interface	**Lex**
Text to speech / speech to text / translation	**Polly** / **Transcribe** / **Translate**
Sentiment or meaning of text	**Comprehend**
Analyze images and videos	**Rekognition**
Extract data from scanned documents	**Textract**
Real-time streaming data	**Kinesis**
ETL, data catalog	**Glue**
SQL directly on S3, serverless	**Athena**
Big data with Spark or Hadoop	**EMR**
Data warehouse	**Redshift**
Search and log analytics	**OpenSearch Service**
BI dashboards	**Amazon Quick**

3.8 Other in-scope services

Application integration: making the parts talk

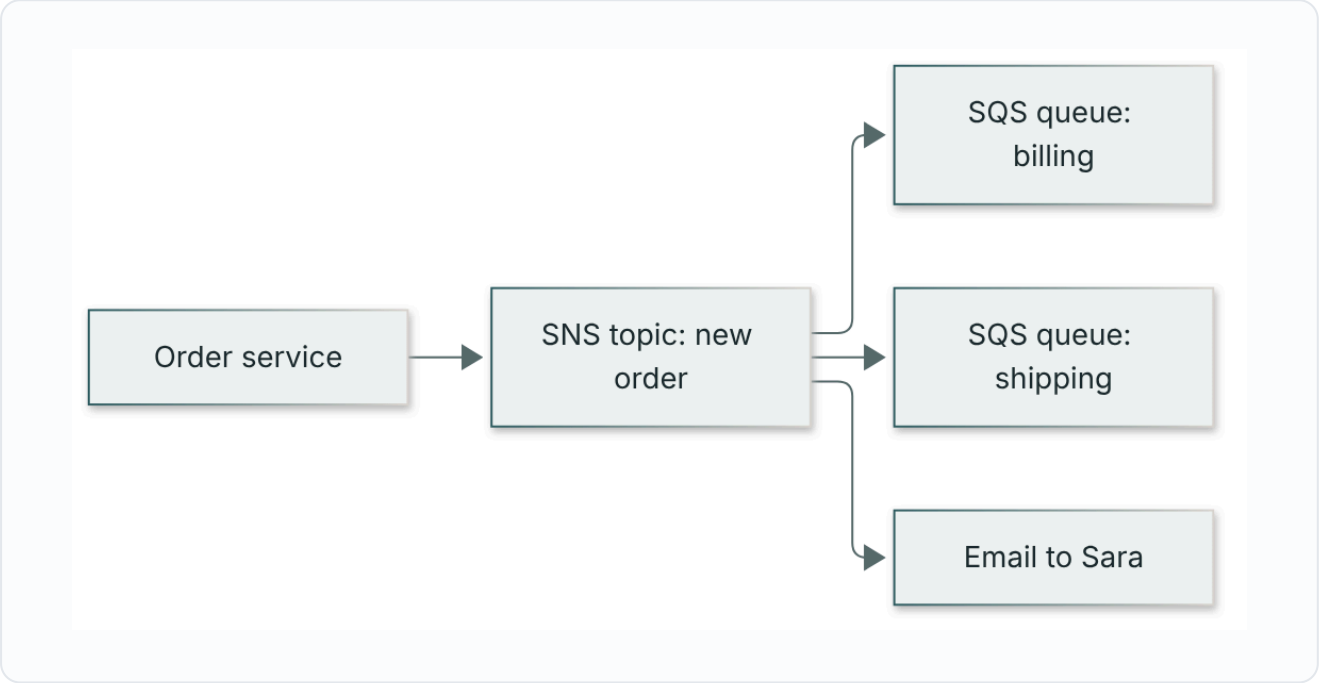
The problem. Modern applications are made of several parts: the website, the order service, the payment service, the email service. If they call each other directly and one is slow or down, everything breaks. The solution is to **decouple** them: put a messaging service between them, so each part can work at its own pace and failures do not spread.

> ANALOGY: A RESTAURANT KITCHEN
>
> **SQS** is the ticket rail: waiters pin orders, cooks take them when they are free; when the kitchen is overloaded, tickets wait but none are lost. **SNS** is the loudspeaker: one announcement is heard by everyone listening at once. **EventBridge** is the receptionist who routes each kind of event to the right person, and rings the bell at fixed times. **Step Functions** is the recipe that says which step comes after which.

Amazon SQS (Simple Queue Service). A **message queue**. One part (the producer) puts messages in the queue; another part (the consumer) **pulls** them when it is ready. Messages wait safely until processed. *Sara's website puts each new order in a queue; the order-processing*

service takes them one by one. On Mother's Day, orders pile up in the queue instead of crashing the system.

Amazon SNS (Simple Notification Service). Publish/subscribe messaging. A message published to a **topic** is **pushed** instantly to **all subscribers**: email addresses, SMS, mobile push notifications, SQS queues, Lambda functions, HTTP endpoints. Used for **alerts and notifications** and for **fan-out** (one event, many receivers). CloudWatch alarms often notify through SNS.



Amazon EventBridge. An **event bus**: it receives events from AWS services, your applications and SaaS partners, and **routes them to targets according to rules** ("when an order over 100 euros is placed, start the VIP workflow"). It also runs **scheduled** tasks ("every night at midnight").

AWS Step Functions. Orchestrates workflows: you define a sequence of steps (call this Lambda function, then wait for approval, then branch, retry on failure), and Step Functions runs it and shows its progress visually.

Amazon MQ. A managed **message broker** for **Apache ActiveMQ and RabbitMQ**. Use it when existing applications already use these standard messaging protocols and must move to AWS **without rewriting** their messaging code; for new applications, SQS and SNS are simpler.

GOOD TO KNOW

Two names from microservice architectures you may meet: **AWS Cloud Map** OUT (service discovery: a registry where services find each other's addresses) and **AWS App Mesh** LEGACY (a service mesh to control traffic between microservices, reaching end of support in 2026).

Business applications

Amazon Connect. A cloud **contact center**: phone and chat customer service, with call routing, recordings, analytics and AI features. Pay per use, set up in minutes. *Sara's customer service line.*

Amazon SES (Simple Email Service). Sends **emails** at scale from your applications: **transactional** emails (order confirmations, password resets) and **marketing** emails (newsletters). It can also receive emails.

> NOTE
> **SNS or SES?** SNS sends **notifications** (short messages to subscribers of a topic: SMS, push, simple email alerts) triggered by events. SES sends **real emails** to customers (HTML, personalized, transactional or marketing). Pinpoint (campaigns) and WorkMail (company mailboxes) are not on the v1.3 list.

Customer enablement

AWS Support, the plans and resources that help customers, is part of this category; it is explained in section 4.3.

Developer tools

These services help developers **build, deploy and troubleshoot** applications, which is the job of a **CI/CD** pipeline (continuous integration and continuous delivery: every code change is automatically built, tested and deployed).

- **AWS CodeBuild**: **compiles** code, runs **tests** and produces ready-to-deploy packages. Serverless, pay per build minute.
- **AWS CodePipeline**: automates the whole **release pipeline**: when code changes, it triggers build, test and deployment stages in order.
- **AWS X-Ray**: **traces requests** as they travel through the parts of a distributed application, showing where errors and slowdowns happen. Used to **troubleshoot**.
- **AWS CLI**: the command line (see 3.1).
- **AWS SAM (Serverless Application Model)**: an open-source framework to **define, test and deploy serverless applications** (Lambda functions, APIs, DynamoDB tables) with short templates that extend CloudFormation.

Out of scope or legacy: CodeDeploy, CodeArtifact, CloudShell, Application Composer OUT; CodeCommit, Cloud9 (no longer available to new customers), CodeStar (discontinued in 2024) LEGACY.

End-user computing

These services **present the output of virtual machines on users' devices**: the user sees and uses software that actually runs in AWS.

- **Amazon WorkSpaces**: full **virtual desktops** (Windows or Linux) in the cloud. Employees reach their desktop from any laptop, tablet or thin client; data stays in AWS. *Useful for remote workers or temporary staff.*
- **Amazon AppStream 2.0**: **streams a single application** (not a whole desktop) to a web browser. *A design company lets clients use a heavy 3D program from a browser.*
- **Amazon WorkSpaces Secure Browser** NEW (formerly WorkSpaces Web): gives employees **secure browser access to internal websites and SaaS applications**, without a full desktop and without data reaching the device.

Frontend web and mobile

AWS Amplify: tools and hosting to **build, deploy and host web and mobile applications** quickly, and connect them to back-end features (authentication, data, storage) without deep cloud expertise.

AWS AppSync LEGACY: managed **GraphQL** APIs for web and mobile apps, with **real-time** updates and offline data synchronization.

Internet of Things

AWS IoT Core: **connects** billions of **devices** (sensors, machines, appliances) to AWS **securely**, receives their messages, and routes data to other services. *Sara's fridges send their temperature every minute; an alert fires if it rises.*

AWS IoT Device Defender OUT audits the security settings of IoT devices and detects abnormal device behavior. **AWS IoT Greengrass** OUT runs AWS code on the devices themselves.

Management and governance, a summary

These services appear throughout the course; here they are together.

Service	In plain words
Amazon CloudWatch	Monitoring: metrics, logs, alarms, dashboards
AWS CloudTrail	Audit log of every API call
AWS Config	Configuration history and compliance rules
AWS Systems Manager	Operate servers at scale: patching, commands, inventory, Parameter Store, Session Manager
AWS Resource Explorer	Search and find your resources across Regions and accounts

Service	In plain words
AWS Resource Access Manager (RAM)	Share resources (subnets, Transit Gateways, license configurations) with other accounts without copying them
AWS Audit Manager LEGACY	Continuously collects **evidence** from your AWS usage to prepare **audits** against standards (PCI DSS, GDPR, SOC 2)
AWS OpsWorks LEGACY	Configuration management with Chef and Puppet; retired in 2024 (Systems Manager is the usual answer today)
AWS Organizations	Manage many accounts: consolidated billing, SCPs
AWS Control Tower	Automated, governed multi-account setup (landing zone)
AWS Service Catalog	A catalog of **approved** products (templates) employees can launch themselves
AWS License Manager	Track software licenses (BYOL)
AWS Compute Optimizer	Rightsizing recommendations from real usage
AWS Health Dashboard	AWS service status and events affecting your resources
AWS Trusted Advisor	Best-practice checks: cost, performance, security, fault tolerance, limits
Service Quotas NEW	View and request increases of service limits (for example, number of instances per Region)
AWS Well-Architected Tool	Review a workload against the six pillars
AWS Auto Scaling	Scaling of resources with demand

Retired, renamed and other services you may still see

Older study material and question banks still mention services that have changed. Know them in one line:

Name you may see	Status and what to remember
Amazon Elastic Transcoder	Converted media files; support ended in November 2025, replaced by AWS Elemental MediaConvert
AWS OpsWorks	Chef and Puppet configuration management; retired in 2024
AWS CodeStar	Project templates for developers; discontinued in 2024
AWS Cloud9	Cloud IDE; no longer available to new customers
Amazon WorkDocs	Document storage and collaboration; ended in 2025
Amazon WorkLink	Secure mobile access to internal websites; replaced by WorkSpaces Secure Browser
Amazon QLDB	Ledger database; ended in 2025
AWS Snowmobile, Snowcone	Retired; Snowball Edge no longer offered to new customers

Name you may see	Status and what to remember
Amazon Q Business	Closed to new customers; Amazon Quick is the recommended assistant for work
Amazon Forecast	Time-series forecasting service; no longer offered to new customers, SageMaker AI is used instead
AWS Ground Station OUT	Satellite ground stations as a service: download and process satellite data without building antennas

Look-alike services: the exam's favourite traps

GOOD TO KNOW

Many questions test pairs of services with similar names or purposes. The table below gathers the classic confusions.

They look alike	The difference
CloudTrail / CloudWatch / Config	**Who** did what (API audit) / **how** it performs (metrics, alarms) / **what** the configuration is and whether it complies
AWS Config / AWS AppConfig OUT	Resource compliance / feature flags and settings for applications
SQS / SNS / EventBridge	Queue, the consumer pulls / message pushed to many subscribers / events routed by rules and schedules
SNS / SES	Notifications / real emails
GuardDuty / Inspector / Trusted Advisor	**Active threats** / **software vulnerabilities** / **best-practice checks** of the account
Shield / WAF / Firewall Manager	DDoS / web exploits such as SQL injection / central management of rules
KMS / CloudHSM / ACM / Secrets Manager	Managed keys / dedicated key hardware / TLS certificates / passwords with rotation
S3 / EBS / EFS / instance store	Objects / disk for one instance / shared Linux files / temporary disk
RDS / Aurora / DynamoDB / Redshift	Managed SQL / high-performance SQL / NoSQL key-value / data warehouse
RDS Multi-AZ / read replica	High availability / read performance
CloudFront / Global Accelerator	Caches content (HTTP) / static IPs and fast routing for any traffic
Direct Connect / VPN	Private dedicated line / encrypted tunnel over the internet
Organizations / Control Tower	Accounts, SCPs, consolidated billing / automated governed setup
IAM Identity Center / Cognito	Employees' single sign-on / your app's customers
Elastic Beanstalk / CloudFormation / Lightsail	Deploy code (PaaS) / infrastructure as code / simple fixed-price server

They look alike	The difference
WorkSpaces / AppStream 2.0	Full desktop / one streamed application
Macie / GuardDuty	Sensitive data in S3 / threats across the account

KEY TAKEAWAYS

- Console for one-time tasks; CLI, SDKs and **CloudFormation** (infrastructure as code) for repeatable ones.
- Region (independent, data stays there) → 2+ AZs (separate data centers, no shared point of failure) → many edge locations (CloudFront, Route 53, Global Accelerator).
- Multi-AZ for high availability; multi-Region for disaster recovery, business continuity, latency and data sovereignty. DR: backup and restore → pilot light → warm standby → multi-site.
- EC2 (control, instance families) · ELB (spread traffic) · Auto Scaling (elasticity) · ECS/EKS/ECR (containers) · Fargate and Lambda (serverless) · Beanstalk (PaaS) · Lightsail (simple) · Batch.
- RDS and Aurora (relational) · DynamoDB (NoSQL) · ElastiCache (cache) · DocumentDB · Neptune (graph) · Redshift (warehouse). Multi-AZ = availability, read replicas = performance. DMS + SCT to migrate.
- VPC with public and private subnets, internet and NAT gateways; security groups (stateful, allow) vs network ACLs (stateless, deny). Transit Gateway, PrivateLink, VPN vs Direct Connect, Route 53, CloudFront, Global Accelerator.
- S3 classes from Standard to Glacier Deep Archive, lifecycle policies; EBS, instance store, EFS, FSx, Storage Gateway, AWS Backup.
- AI services by task, SageMaker AI to build models, Amazon Q; Kinesis, Glue, Athena, EMR, Redshift, OpenSearch, Quick for analytics.
- SQS (queue), SNS (notifications), EventBridge (events), Step Functions (workflows), Connect, SES, WorkSpaces, AppStream, Amplify, IoT Core, CodeBuild, CodePipeline, X-Ray.

PART 4 · DOMAIN 4

Billing, Pricing, and Support

12% of the scored content

How AWS charges, how to pay less, the tools that keep the bill under control, and where to get help. A small domain with easy points if you understand the logic of each option.

IN THIS PART

4.1 Compare AWS pricing models

4.2 Understand resources for billing, budget, and cost management

4.3 Identify AWS technical resources and AWS Support options

OUR STORY THROUGH THIS COURSE

Sara's shop runs well, but her accountant asks three questions: why does the bill change every month, how can we pay less, and who do we call if something breaks on a Saturday night?

4.1 Pricing models

How AWS charges: three principles

1. **Pay as you go.** You pay for what you use, for as long as you use it, with no upfront payment, no long-term contract and no termination fee. Stop using a resource and you stop paying.
2. **Pay less when you reserve.** If you **commit** to using a certain amount for **1 or 3 years**, AWS gives you a large discount, because it can plan its capacity.
3. **Pay less per unit when you use more.** Some services have **volume tiers**: for example, the price per GB stored in S3 drops as your total storage grows.

What you actually pay for. Almost every AWS bill comes from three things:

- **Compute**: the time your servers, containers or functions run (per second, per hour, or per request and millisecond for Lambda).
- **Storage**: the amount of data you keep, per GB per month, depending on the storage class.
- **Data transfer out**: data **leaving** AWS to the internet or to another Region.

GOOD TO KNOW

A useful habit: there are about **730 hours in a month**. Multiply an hourly price by 730 to estimate the monthly cost. A cache node at about 0.20 dollars per hour, left running and forgotten, already costs around 150 dollars a month. That is why you create a budget on day one (section 4.2).

Buying EC2 capacity: the seven options

IN PLAIN WORDS

The same EC2 server can be paid for in several ways. The cheapest option depends on **how long** you will use it, **how predictable** your usage is, and whether your work **can be interrupted**.

ANALOGY: A HOTEL

On-Demand is walking in and paying the full nightly rate: total freedom, highest price. **Reserved Instances** is booking the same type of room for one or three years at a big discount. **Savings Plans** is promising to spend a fixed amount per hour at the hotel chain, and using any room you like. **Spot** is taking unsold rooms at up to 90% off, but you may be asked to leave with two minutes' notice. A **Dedicated Host** is renting a whole building for yourself. A **Capacity Reservation** is paying to hold a room on a specific date, whether you come or not.

On-Demand Instances. Pay by the second (Linux) or hour, **no commitment**, start and stop whenever you want. The most flexible and the most expensive per hour. **When**: short-term, unpredictable or spiky workloads, development and testing, new applications whose usage you do not know yet, workloads that cannot be interrupted.

Savings Plans. You commit to a **consistent amount of spending per hour** (for example 10 dollars per hour) for **1 or 3 years**, and receive up to about **72% discount** on usage up to that amount. Two main kinds:

- **Compute Savings Plans**: the most **flexible**: the discount applies to EC2 of **any family, size, Region and operating system**, and also to **AWS Fargate and AWS Lambda**. You can change instances freely.
- **EC2 Instance Savings Plans**: bigger discount, but tied to **one instance family in one Region** (any size, OS within it).

When: steady, predictable usage, when you want a big discount **and** flexibility.

Reserved Instances (RIs). You commit to a **specific instance configuration** (instance type, Region, operating system, tenancy) for **1 or 3 years**, and get up to about **72% discount**. Reserved Instances also exist for **RDS, ElastiCache, Redshift** and other services.

- **Payment options**: **All Upfront** (biggest discount), **Partial Upfront**, **No Upfront** (smallest discount). A 3-year term gives more discount than 1 year.

- **Standard RIs**: the **largest discount**. You can modify some attributes (the AZ, the instance size within the same family) and you can **sell** unused Standard RIs on the **Reserved Instance Marketplace**.
- **Convertible RIs**: smaller discount, but you can **exchange** them for RIs of a **different family, operating system or tenancy**, as long as the new value is equal or greater.

When: steady-state workloads that run all the time with a known configuration, for example a database running 24 hours a day.

Reserved Instance flexibility. A Regional RI applies its discount automatically to **any size within the same instance family** in that Region (for example, one `large` reservation can cover two `medium` instances), and Convertible RIs can be exchanged. That is what the guide calls RI flexibility.

Reserved Instances in AWS Organizations. When several accounts are grouped in an organization with consolidated billing, **RI and Savings Plans discounts are shared across all accounts**: if account A does not use all its reservations, the discount applies to matching usage in account B. Discount sharing can be turned off for specific accounts.

Spot Instances. Use AWS's **spare, unused capacity** at up to about **90% discount**. The catch: when AWS needs the capacity back, it can **interrupt** your instance with a **two-minute warning**. **When**: work that **can stop and restart without harm**: batch processing, big data analysis, image and video rendering, scientific simulations, CI/CD test runs, stateless web servers in a mixed fleet. **Never** for critical databases or work that cannot be interrupted. A **Spot Fleet** (or EC2 Fleet) launches and keeps a target capacity from a mix of instance types and purchasing options, choosing the cheapest available capacity automatically.

Dedicated Hosts. A **whole physical server dedicated to you**, where you can see the **sockets and cores** and control where your instances are placed. **When**: **bring your own license (BYOL)** for software licensed per socket or per core (Windows Server, SQL Server, Oracle), and strict compliance needs. The most expensive option. Can be paid On-Demand or reserved.

Dedicated Instances. Instances that run on hardware **dedicated to your account** (no other customer on the same machine), but without visibility or control of the physical server. **When**: compliance requires hardware isolation, without the licensing needs of Dedicated Hosts.

Capacity Reservations. Reserve EC2 capacity in a specific Availability Zone for any duration, so you are sure the instances are available when needed. No commitment and **no discount by themselves**: you pay the On-Demand price whether you use the capacity or not, and you can combine them with Savings Plans or Regional RIs to get a discount. **When**: a critical event (Black Friday), disaster recovery, or regulatory needs to guarantee capacity.

Option	Commitment	Discount	Best for
On-Demand	None	None	Short, unpredictable, dev/test, cannot be interrupted

Option	Commitment	Discount	Best for
Savings Plans	1 or 3 years, $ per hour	Up to ~72%	Steady usage with flexibility (Compute SP covers Fargate and Lambda)
Reserved Instances	1 or 3 years, specific configuration	Up to ~72%	Steady, predictable workloads; Standard (sellable) or Convertible (exchangeable)
Spot	None	Up to ~90%	Fault-tolerant, interruptible jobs
Dedicated Hosts	On-Demand or reserved		BYOL per socket/core, compliance
Dedicated Instances	On-Demand or reserved		Hardware isolation
Capacity Reservations	None	None alone	Guaranteed capacity in one AZ

EXAMPLE

Sara's shop: her database runs all year with the same size → **Reserved Instance** (or a Savings Plan). Her web servers grow and shrink → a **Compute Savings Plan** for the base, **On-Demand** for peaks. Her nightly job that resizes photos can be restarted if stopped → **Spot**.

Data transfer costs

IN PLAIN WORDS

Moving data also costs money, but not in every direction.



- **Data coming into AWS** from the internet: **free** (AWS wants your data to come in).
- **Data going out to the internet**: **charged** per GB, with a small free allowance each month and lower prices at high volumes.
- **Between two Regions**: **charged**.
- **Between Availability Zones** in the same Region: a **small charge** per GB.
- **Within the same AZ** using private IP addresses: **free**.
- **Amazon CloudFront** and **AWS Direct Connect** can **reduce** the cost of data going out.

EXAMPLE

Sara uploads 100 GB of photos to S3: free. Customers download 500 GB of pages and photos in a month: she pays for data out, less if served through CloudFront. She copies her backups to another Region: she pays inter-Region transfer.

Storage pricing

Storage prices depend on: the **amount** stored (per GB per month), the **storage class** (S3 Standard is more expensive to store than Glacier, but cheaper to read), the number of **requests**, **retrieval fees** for infrequent and archive classes, **minimum storage durations** (30, 90 or 180 days), and **data transfer** out. For EBS, the **volume type** (fast SSD or cheaper HDD) and its size and performance matter.

Free things and the Free Tier

- **Free services**: IAM, VPC, AWS Organizations and consolidated billing, Auto Scaling, CloudFormation and Elastic Beanstalk. For CloudFormation, Beanstalk and Auto Scaling you pay only for the resources they create.
- **AWS Free Tier**: lets new customers try AWS at no cost within limits: offers that are **always free** (for example a monthly number of Lambda requests or a small amount of DynamoDB storage), offers **free for a limited period** after sign-up, and short **trials** of some services. AWS has changed the program for new accounts recently (including credit-based plans), so check the current terms on the AWS Free Tier page.
- **Promotional credits**: AWS credits (for example from events, startup programs or training) are applied to your bill like money.

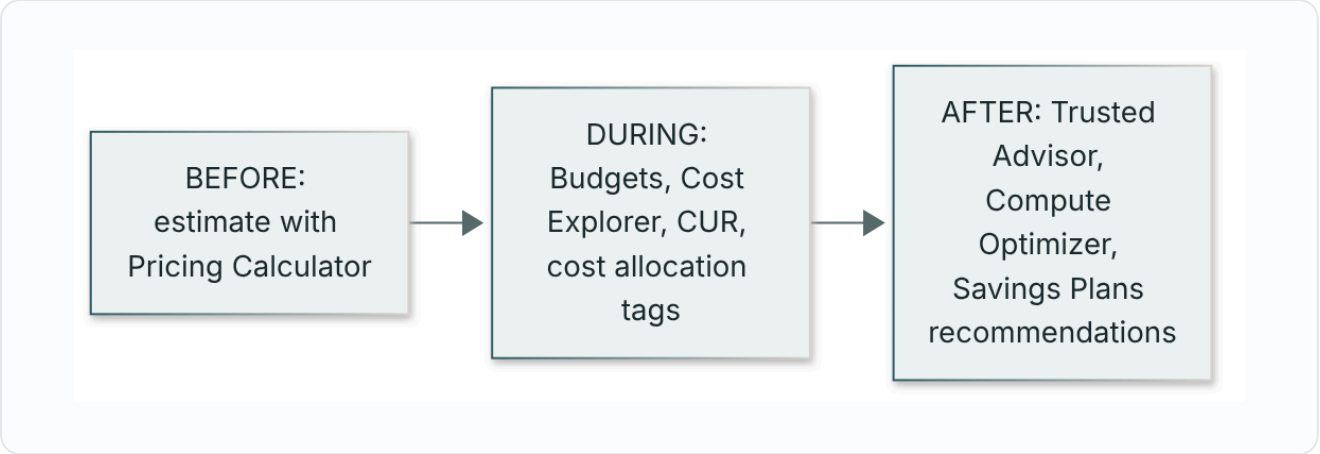
ON THE EXAM

"Unpredictable, short-term, cannot be interrupted" = **On-Demand**. "Steady 1 to 3 years, flexible across instance families, includes Lambda and Fargate" = **Compute Savings Plans**. "Steady, specific instance, biggest discount" = **Standard Reserved Instances**; "need to change family later" = **Convertible**. "Cheapest, can be interrupted" = **Spot**. "Per-core licenses" = **Dedicated Hosts**. "Guarantee capacity in an AZ" = **Capacity Reservations**. "Data into AWS" = **free**. "RI discounts in an organization" = **shared across accounts**.

4.2 Billing, budgets and cost management

IN PLAIN WORDS

AWS gives a set of free or low-cost tools to manage spending. Think of three moments: **before** building (estimate), **while** running (watch, alert, understand), and **after** (optimize).



AWS Pricing Calculator: estimate before you build. A free website (no account needed) where you describe the architecture you plan, for example "two web servers of this size, a 100 GB database, 1 TB of S3, 500 GB of data out", and get an **estimated monthly cost**. You can save and share the estimate. Use it to compare options and to build budgets or migration business cases. For programs, the **AWS Price List API** gives AWS prices programmatically.

AWS Budgets: get alerted. You set a **budget** for cost, usage, or Reserved Instance and Savings Plans coverage, for example "300 dollars per month". AWS Budgets sends **alerts** (by email or through SNS) when the **actual** spending, or the **forecasted** spending for the month, passes thresholds you choose (for example 80% and 100%). Budgets can also trigger **actions**, such as applying a policy that stops creating new resources.

AWS Cost Explorer: understand and forecast. An interactive tool with **charts** of your costs and usage over time: by service, by account, by Region, by tag. It shows up to the last 12 months (more on request), **forecasts** future spending, and gives **recommendations**: rightsizing of underused instances, and which Reserved Instances or Savings Plans to buy. Use it to answer "what did we spend on, and why did it change?".

> GOOD TO KNOW
>
> **AWS Billing Conductor** OUT creates **custom pricing and billing views** for groups of accounts, used by resellers and by companies that charge back costs internally with their own rates.

AWS Cost and Usage Report (CUR): every detail. The **most comprehensive and detailed** billing data: one line per resource, per hour or day, with prices, discounts and tags. It is delivered as files into an **S3 bucket**, to be analyzed with tools such as Amazon Athena or Amazon Quick. Finance teams and large companies use it for detailed analysis and chargeback.

Cost allocation tags: split the bill. A **tag** is a label made of a **key** and a **value** that you attach to AWS resources, for example `project = website`, `team = marketing`, `environment = test`. Once **activated** in the Billing console as **cost allocation tags**, tags appear in Cost Explorer and the Cost and Usage Report, so you can see the cost **per project, team or environment**. Two types:

- **AWS-generated tags**: created by AWS, such as `aws:createdBy` (who created the resource).

- **User-defined tags**: created by you, such as `project`.

Both must be activated before they show in billing reports.

AWS Organizations and consolidated billing. A company often has **many AWS accounts** (one per team, per project, per environment), which isolates them for security. AWS Organizations groups them, with one **management account** that receives **one single bill** for all **member accounts**: this is **consolidated billing**, a free feature. Benefits:

- **One bill** and one payment method for everything, with costs still visible per account.
- **Combined usage for volume discounts**: the usage of all accounts is added together to reach cheaper price tiers sooner.
- **Shared Reserved Instance and Savings Plans discounts** across accounts.

> EXAMPLE
> Sara now has three accounts: production, test and accounting. Each uses 40 TB of S3. Separately, none reaches the next cheaper tier; combined under consolidated billing, 120 TB does, and every account benefits from the lower price.

AWS Cost Anomaly Detection uses machine learning to spot **unusual spending** (a sudden spike) and alerts you.

Billing support. Questions about bills and payments can be sent to AWS through the **Support Center**; billing and account support is included in **every** Support plan, even Basic.

> GOOD TO KNOW
> Some services have expensive defaults, and a beginner can receive a surprise bill. Create a budget with an email alert first. If it happens to you, open a billing case in the Support Center and explain: AWS often refunds a first honest mistake.

> ON THE EXAM
> "Estimate the cost before building or migrating" = **Pricing Calculator**. "Alert when cost exceeds or is forecasted to exceed a threshold" = **Budgets**. "Visualize, analyze past costs, forecast, rightsizing recommendations" = **Cost Explorer**. "Most detailed, line-item billing data in S3" = **Cost and Usage Report**. "See costs per department or project" = **cost allocation tags** (activated). "One bill for many accounts, volume discounts" = **consolidated billing** in **Organizations**.

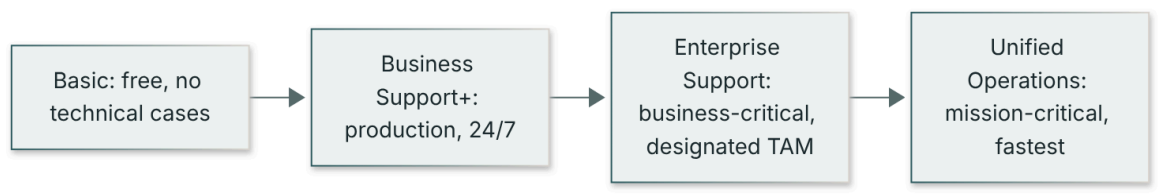
4.3 Technical resources and AWS Support

AWS Support plans

IN PLAIN WORDS

Every AWS account gets **Basic Support** for free: customer service for **account and billing** questions, documentation, whitepapers and the **re:Post** community. But Basic Support **does not include technical support cases**: you cannot ask an AWS engineer to help with a technical problem. For that you need a **paid plan**. The higher the plan, the faster the response and the more proactive help you get.

The exam guide v1.3 uses **new plan names**. Learn them:



Basic Support (free, all customers). Customer service for billing and account questions, documentation, whitepapers, re:Post, the AWS Health Dashboard for your account, and the **core** Trusted Advisor checks.

AWS Business Support+ NEW. The **minimum plan recommended for production workloads**. Technical support **24/7** by phone, chat and email from AWS engineers, unlimited cases, help with third-party software, the **full set** of Trusted Advisor checks, AI-powered assistance, and response within **30 minutes** when a critical system is down (one hour when a production system is down).

AWS Enterprise Support. For **business-critical** workloads. Everything in Business Support+, plus a **designated Technical Account Manager (TAM)**, response within **15 minutes** for business-critical system down, consultative architecture reviews, security incident response included, and concierge billing help.

AWS Unified Operations NEW. For **mission-critical** workloads that need the highest resilience. The **fastest response, within 5 minutes**, a team of designated specialists with continuous architecture and resiliency reviews, incident detection and response included.

	Basic	Business Support+	Enterprise Support	Unified Operations
For	Everyone	Production (minimum recommended)	Business-critical	Mission-critical
Technical cases	**No**	Yes, 24/7	Yes, 24/7	Yes, 24/7
Critical system down		< 30 min	**< 15 min**	**< 5 min**
Production system down		< 1 h	< 1 h	< 1 h

	Basic	Business Support+	Enterprise Support	Unified Operations
Technical Account Manager	No	No	**Designated**	**Designated**
Trusted Advisor	Core checks	**All checks**	All + Priority	All + Priority
Price	Free	From about $29/month verify	From about $5,000/month verify	verify

The old names (you may still meet them in some questions or older material; these plans end on 1 January 2027): **Developer** (Developer Support: business-hours email, for testing) and **Business** (24/7, production) are replaced by **Business Support+**; **Enterprise On-Ramp** (pool of TAMs) is replaced by **Enterprise Support**; **Enterprise** = Enterprise Support.

Technical Account Manager (TAM). A named AWS technical expert who knows your environment, gives **proactive** guidance (architecture, cost, operations), coordinates access to AWS specialists, and helps during incidents. Only in **Enterprise Support** and **Unified Operations**.

Infrastructure Event Management (IEM). AWS experts help you **prepare and run a planned event** (a product launch, a big sales day, a migration): architecture review, capacity planning and close support during the event. It is offered with the higher Support plans.

Tools that watch your environment

AWS Trusted Advisor inspects your AWS account and gives **recommendations** to follow best practices. Its main categories:

Category	Example of check
Cost optimization	Idle or underused EC2 instances, unassociated Elastic IP addresses, idle load balancers
Performance	Overused instances, CloudFront configuration
Security	MFA on the root user, security groups open to the world, public S3 buckets, exposed access keys
Fault tolerance	No EBS snapshots, instances in a single AZ, RDS without backups
Service limits (quotas)	Usage close to a service limit

(There is also an operational excellence category.) Basic Support gets the core checks; Business Support+ and higher get all checks and programmatic access.

AWS Health Dashboard (formerly the Personal Health Dashboard) shows two things: the **general status of AWS services** in every Region, and, more importantly, **events that affect**

your own resources: scheduled maintenance on one of your instances, an issue in the AZ where your database runs, a certificate about to expire. The **AWS Health API** lets you receive these events in your own tools and automate reactions.

Where to find help and information

Resource	Use it for
AWS Documentation	Official user guides and API references for every service
AWS Whitepapers and guides	In-depth documents, such as the Well-Architected Framework or security best practices
AWS Blogs and **What's New**	News, announcements, tutorials
AWS Prescriptive Guidance	Proven strategies, guides and patterns for migration and modernization projects
AWS Knowledge Center	Answers to the **most frequent questions** received by AWS Support
AWS re:Post	**Community** questions and answers, moderated with AWS experts (replaced the old forums)
AWS Architecture Center	Reference architectures and diagrams
AWS Support Center	Where you **open and follow support cases**, technical or billing
AWS Online Tech Talks and AWS events	Free webinars and talks about services and best practices
AWS Trust & Safety team	Where you **report abuse** coming from AWS resources: spam, attacks, malware, phishing hosted on AWS

Partners, Marketplace and expert help

The AWS Partner Network (APN). Thousands of companies work with AWS to help customers. Two main kinds:

- **Independent software vendors (ISVs)**: companies that **sell software** that runs on or integrates with AWS (security tools, databases, monitoring), often through AWS Marketplace.
- **System integrators and consulting partners**: companies that **provide services**: design architectures, run migrations, manage environments, train teams.

Benefits of being an AWS Partner: access to **partner training and certification**, **partner events**, marketing support and funding, technical support, and **partner volume discounts**.

AWS Marketplace is a **digital catalog** of thousands of software products, data products and professional services from third-party vendors that you can **find, buy and deploy** in a few clicks, often as ready-made AMIs or SaaS. Key services it offers: **cost management** (usage-based or annual pricing, charges on your **AWS bill**, private offers with negotiated prices),

governance (control which products employees may buy), and **entitlement** (license management and tracking).

Technical assistance from AWS itself:

- **AWS Professional Services**: a global team of AWS experts you can hire to help with large projects, such as a data center migration or a transformation program.
- **AWS solutions architects**: AWS technical experts who give **architecture guidance** to customers and help them design solutions with best practices.

Finding and using partners: the **AWS Partner Solutions Finder** searches qualified partners by need, industry and location. **AWS Partner Solutions** (formerly AWS Quick Starts) are automated reference deployments of popular software on AWS, built by AWS and partners. **AWS IQ** OUT is a marketplace to hire AWS Certified freelancers and consulting firms for short projects. With **AWS Partner-Led Support**, some partners that resell AWS also provide first-line technical support to their customers, with AWS behind them.

Service level agreements (SLAs). Each AWS service publishes an **SLA**: a commitment to a monthly availability percentage (for example 99.99%). If AWS misses it, customers receive **service credits** on their bill.

> ON THE EXAM
> "Minimum plan recommended for production" = **Business Support+**. "Designated TAM" = **Enterprise Support** or **Unified Operations**. "Fastest response, mission-critical" = **Unified Operations** (< 5 min). "No technical support cases" = **Basic**. "Best-practice checks: cost, security, fault tolerance, limits" = **Trusted Advisor**. "Events affecting my resources" = **AWS Health Dashboard** (API: **AWS Health API**). "Community answers" = **re:Post**. "Frequent support questions" = **Knowledge Center**. "Migration patterns" = **Prescriptive Guidance**. "Report abuse" = **Trust & Safety**. "Buy third-party software billed on the AWS bill" = **Marketplace**. "Software vendor" = **ISV**; "consulting and migration partner" = **system integrator**. "AWS experts for a big project" = **Professional Services**.

KEY TAKEAWAYS

- Pay as you go, pay less when you reserve, pay less per unit when you use more. You pay for compute, storage and data transfer **out**; data **in** is free.
- On-Demand (flexible) · Savings Plans and RIs (1 or 3 years, up to ~72%; Standard vs Convertible; shared in Organizations) · Spot (up to ~90%, interruptible) · Dedicated Hosts (BYOL) · Dedicated Instances · Capacity Reservations (guaranteed capacity in an AZ).
- Pricing Calculator (before) · Budgets (alerts) · Cost Explorer (analyze, forecast, recommend) · CUR (line-item detail in S3) · cost allocation tags (activate them) · Organizations (one bill, combined volume, shared discounts).
- Support v1.3: Basic (no technical cases) → Business Support+ (production, < 30 min) → Enterprise Support (TAM, < 15 min) → Unified Operations (mission-critical, < 5 min). Old names map to them.
- Trusted Advisor checks the account; Health Dashboard shows events affecting you. re:Post, Knowledge Center, Prescriptive Guidance, documentation, Support Center, Trust & Safety.
- Partners: ISVs sell software, system integrators provide services; Marketplace sells third-party products on your AWS bill; Professional Services and solutions architects are AWS's own experts.

PART 5

Exam day

Strategy, key words, checklist and self-test

How to read questions, a key-word map to answer fast, the checklist for the last hour, and 30 questions to test yourself.

5.1 How to answer

GOOD TO KNOW

Aim for **about 85%** on practice exams before booking: the real exam usually feels about 10 points harder. There is no reason to target 100%: pass and move on to the next certification.

1. Read the **last sentence first**: what exactly is asked (cheapest? most available? least effort?).
2. Read the scenario and underline the key words.
3. **Eliminate** the two answers that are clearly wrong, then choose between the remaining two.
4. For multiple response, select **exactly** the number requested.
5. When unsure, prefer the **managed or serverless** service: AWS likes answers where AWS does the work.
6. Never leave a blank: wrong answers cost nothing.
7. A strange, very hard question may be one of the 15 unscored ones: do your best and move on.
8. About 80 seconds per question: flag long ones and come back at the end.

5.2 Key word → answer

If you read	Think
audit, who made the API call	CloudTrail
metrics, alarm, billing alarm	CloudWatch
configuration history, compliance rules	Config
compliance report (SOC, PCI, ISO)	Artifact
DDoS	Shield

If you read	Think
SQL injection, cross-site scripting	WAF
threat detection, compromised	GuardDuty
vulnerabilities, CVE	Inspector
personal data in S3	Macie
investigate root cause	Detective
single security dashboard	Security Hub
employees, single sign-on, many accounts	IAM Identity Center
sign-in for app customers	Cognito
rotate database passwords	Secrets Manager
encryption keys	KMS (dedicated hardware: CloudHSM)
HTTPS certificate	Certificate Manager
estimate before building	Pricing Calculator
alert on cost	Budgets
analyze and forecast cost	Cost Explorer
most detailed billing	Cost and Usage Report
cheapest, can be interrupted	Spot
steady 1 to 3 years, flexible	Savings Plans
per-core licenses	Dedicated Hosts
guarantee capacity in an AZ	Capacity Reservation
decouple	SQS
notify many subscribers	SNS
events, schedules	EventBridge
workflow	Step Functions
SQL on files in S3	Athena
real-time streaming	Kinesis
ETL	Glue
data warehouse	Redshift
NoSQL, millisecond, any scale	DynamoDB
cache in memory	ElastiCache
MongoDB	DocumentDB

If you read	Think
graph, relationships	Neptune
shared Linux files	EFS
Windows file share	FSx for Windows File Server
disk for an EC2 instance	EBS
cheapest archive	S3 Glacier Deep Archive
unknown access pattern	S3 Intelligent-Tiering
on-premises apps using cloud storage	Storage Gateway
content close to users	CloudFront
DNS, domain	Route 53
connect many VPCs	Transit Gateway
private connection to a service	PrivateLink
dedicated private line	Direct Connect
encrypted tunnel over the internet	Site-to-Site VPN
infrastructure as code	CloudFormation
upload code, AWS runs it	Elastic Beanstalk
simple server, fixed monthly price	Lightsail
containers without servers	Fargate
Kubernetes	EKS
code triggered by events	Lambda
elasticity	Auto Scaling
chatbot	Lex
text to speech / speech to text	Polly / Transcribe
build your own ML model	SageMaker AI
generative AI assistant	Amazon Q
contact center	Connect
send emails	SES
virtual desktop	WorkSpaces
business case for migration	Migration Evaluator
move servers as they are	Application Migration Service (rehost)
migrate a database	DMS (+ SCT to convert the schema)

If you read	Think
best-practice checks on the account	Trusted Advisor
limits of a service	Service Quotas
designated TAM	Enterprise Support or Unified Operations
report abuse	Trust & Safety
community answers	re:Post

5.3 Last-hour checklist

- The 6 advantages of cloud, the 6 Well-Architected pillars, the 6 CAF perspectives and 4 benefits, the 7 Rs
- Shared responsibility for EC2, RDS, Lambda, S3; root-only tasks; least privilege
- CloudTrail vs CloudWatch vs Config; the security services; security group vs network ACL
- Region, AZ, edge location; 4 factors to choose a Region; DR strategies with RPO and RTO
- S3 classes in order; EBS vs EFS vs instance store; database per need
- EC2 buying options; data transfer rules; cost tools; the 4 Support plans and the old names

5.4 Test yourself

> **NOTE**
> Cover the answers in italics, answer each question aloud, then check. Every wrong answer points to a section to re-read.

1. A startup does not want to buy hardware and wants to pay monthly for what it uses. Which advantage? *Trade fixed expense for variable expense.*
2. Which pillar includes "automatically recover from failure"? *Reliability.*
3. A company wants to move servers to AWS as fast as possible without changing them. Strategy? *Rehost.*
4. Which CAF perspective covers training and culture? *People.*
5. On Amazon EC2, who patches the operating system? *The customer.* On Amazon RDS, who patches the database engine? *AWS.*
6. An EC2 instance must read files in S3. What is the secure way? *Attach an IAM role to the instance.*
7. Which service shows who deleted an S3 bucket yesterday? *CloudTrail.*
8. An auditor wants AWS's PCI DSS report. Where? *AWS Artifact.*

9. You must block one malicious IP address for a whole subnet. What? *A network ACL with a deny rule.*
10. The website suffers SQL injection attempts. Service? *AWS WAF.*
11. Which service finds personal data stored in S3? *Amazon Macie.*
12. Employees should sign in once to reach 15 AWS accounts. Service? *IAM Identity Center.*
13. Your application must stay up if one data center fails. Design? *Instances in several Availability Zones behind a load balancer.*
14. Which DR strategy keeps a small but complete copy running in another Region? *Warm standby.*
15. Traffic varies a lot during the day. Which service adds and removes instances automatically? *EC2 Auto Scaling.*
16. Run code when a file is uploaded, without managing servers? *AWS Lambda.*
17. Logs must be kept 7 years for compliance and are almost never read. Cheapest storage? *S3 Glacier Deep Archive.*
18. Several Linux instances need the same shared files. Service? *Amazon EFS.*
19. A game needs a NoSQL database with millisecond latency at any scale. Service? *Amazon DynamoDB.*
20. Managers want sales trends over five years from huge data. Service? *Amazon Redshift.*
21. Users in Asia complain that images load slowly from your European website. Service? *Amazon CloudFront.*
22. A batch job can be interrupted and must be as cheap as possible. Purchasing option? *Spot Instances.*
23. A database runs 24/7 for the next 3 years. Cheapest option? *Reserved Instances (or a Savings Plan).*
24. Finance wants an email when the monthly forecast exceeds the budget. Tool? *AWS Budgets.*
25. Which is the minimum Support plan recommended for production workloads? *Business Support+.*
26. Which Support plans include a designated Technical Account Manager? *Enterprise Support and Unified Operations.*
27. Where do developers ask questions to the community? *AWS re:Post.*
28. Is data transferred into AWS from the internet charged? *No.*
29. Which service gives best-practice recommendations on cost, security, performance, fault tolerance and limits? *AWS Trusted Advisor.*
30. A company wants a cost estimate of its new architecture before building it. Tool? *AWS Pricing Calculator.*

Then practise with the 70 questions of this guide, the official AWS Practice Question Set (free on Skill Builder) and a free practice exam. Book the exam when you score 85% or more.